

INTELLIGENT EMOTION RECOGNITION AND RECOMMENDATION SYSTEM

A project report submitted in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING (AI & ML)

by

K. SAI PRADEEP	22BFA33049
B. NARASIMHULU	22BFA33027
M. ABILASHA	22BFA33055
B.SREEKAR	22BFA33020
A. PRAVEEN KUMAR	23BFA33L01

Under the guidance of

Dr. P. Lokesh Kumar Reddy, M. Tech, Ph.D

Associate professor



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI & ML)
SRI VENKATESWARA COLLEGE OF ENGINEERING
(AUTONOMOUS)

**(Approved by AICTE, New Delhi & Permanently Affiliated to JNTUA, Ananthapuramu
Accredited by NBA, New Delhi & NAAC with 'A' grade)
Karakambadi Road, TIRUPATI – 517507**

2022 - 2026

SRI VENKATESWARA COLLEGE OF ENGINEERING

(AUTONOMOUS)

(Approved by AICTE, New Delhi & Permanently Affiliated to JNTUA, Ananthapuramu
Accredited by NBA, New Delhi & NAAC with 'A' Grade)
Karakambadi Road, TIRUPATI – 517507.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI & ML)



CERTIFICATE

This is to certify that the project work entitled, **“INTELLIGENT EMOTION RECOGNITION AND RECOMMENDATION SYSTEM”** is a bonafide record of the project work done and submitted by

B. NARASIMHULU	22BFA33027
M. ABILASHA	22BFA33055
K. SAI PRADEEP	22BFA33049
B.SREEKAR	22BFA33020
A. PRAVEEN KUMAR	23BFA33L01

under my guidance and supervision for the partial fulfillment of the requirements for the award of B.Tech degree in **COMPUTER SCIENCE AND ENGINEERING (AI & ML)**. This project is the result of our own effort and that it has not been submitted to any other University or Institution for the award of any degree or diploma other than specified above

GUIDE

HEAD OF THE DEPARTMENT

External Viva-Voce Exam held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

We hereby declare that the project entitled **“INTELLIGENT EMOTION RECOGNITION AND RECOMMENDATION SYSTEM”**, done by us under the esteemed guidance of **Dr. P. Lokesh Kumar Reddy**, Associate professor and is submitted in partial fulfillment of the requirements for the award of the Degree of **Bachelor of Technology in Computer Science and Engineering (AI & ML)**.

This report is the result of our own effort, and it has not been submitted to any other University or Institution for the award of any degree or diploma other than specified above.

Date:

Place:

B. NARASIMHULU	22BFA33027
M. ABILASHA	22BFA33055
K. SAI PRADEEP	22BFA33049
B.SREEKAR	22BFA33020
A. PRAVEEN KUMAR	23BFA33L01

ACKNOWLEDGEMENT

We are thankful to our guide **Dr. P. Lokesh Kumar Reddy, Associate professor, Department of CSE (AI&ML)** for her valuable guidance and encouragement. Her helping attitude and suggestions have helped in the successful completion of the project report.

We would like to express our gratefulness and sincere thanks to **Dr. R. Swathi , Professor & HOD, Department of CSE (AI&ML)**, for her kind help and encouragement during the course of study and in the successful completion of the project work.

We have great pleasure in expressing our hearty thanks to our beloved Principal **Dr. Vijaya Gunturu** for spending his valuable time with us to complete this project.

Successful completion of any project cannot be done without proper support and encouragement. We sincerely thank the management for providing all the necessary facilities during the course of study.

We would like to thank our parents and friends, who have the greatest contributions in all our achievements, for the great care and blessings in making us successful in all our endeavors.

B. NARASIMHULU	22BFA33027
M. ABILASHA	22BFA33055
K. SAI PRADEEP	22BFA33049
B.SREEKAR	22BFA33020
A. PRAVEEN KUMAR	23BFA33L01

ABSTRACT

Mental health challenges affect a significant portion of the global population, yet many individuals lack access to timely, affordable, and stigma-free support. Most existing mental health and journaling applications remain largely passive: they capture entries but fail to extract nuanced emotional insights, offer only generic suggestions, and often rely on infrastructure that does not prioritize user privacy. The **Intelligent Emotion Recognition and Reminder System (IERRS)** addresses these limitations by providing an AI-driven, privacy-aware platform that analyses users' daily journals and proactively supports their emotional well-being through personalised interventions and scheduled self-care reminders.

IERRS accepts both text and voice-based journal entries, transcribes spoken input using a speech-to-text engine, and preprocesses the resulting text for emotion analysis. A transformer-based NLP model (DistilBERT), fine-tuned for emotion classification, detects a range of emotional states (such as joy, sadness, anger, fear, and surprise) and assigns confidence scores to each prediction. Based on these detected emotions and their temporal patterns, the system generates personalised wellness suggestions grounded in cognitive-behavioural therapy and mindfulness practices, and can automatically schedule reminders via calendar integration when persistent negative emotions are observed. All journal data is stored securely using a privacy-first design with encryption and structured emotional histories.

The system is implemented as a full-stack web application using a **React** frontend and **FastAPI** backend, with **SQLModel** for database management and support for real-time dashboards that visualise emotion distributions, trends, and mood patterns over time. Evaluation metrics such as accuracy, F1-score, inference latency, and user feedback ratings demonstrate that the solution can provide responsive and reliable emotion detection suitable for everyday use. By combining advanced natural language processing, behavioural psychology, and automated scheduling, IERRS offers a practical, scalable, and user-centric approach to proactive emotional self-awareness and personalised mental wellness support.

TABLE OF CONTENTS

	Description	Page No.
	ABSTRACT	i
	TABLE OF CONTENTS	ii
	LIST OF FIGURES	iv
	LIST OF TABLES	v
	ABBREVIATIONS	vi
Chapter No.	Description	Page No.
1	INTRODUCTION	1-2
2	PROJECT DESCRIPTION	3-9
2.1	Problem Definition	3-4
2.2	Project Details	4-7
2.3	Scope of the Project	8-9
3	SYSTEM ANALYSIS	10-13
3.1	Existing System	10
3.1.1	Disadvantages of Existing Systems	10
3.2	Proposed System	11-12
3.2.1	Advantages of the Proposed System	12-13
4	ENVIRONMENTS	14-19
4.1	Software Requirements	14
4.2	Hardware Requirements	14-15
4.3	Software Features and Dependencies	15-19
5	SYSTEM DESIGN	20-32
5.1	System Architecture	20-22
5.2	Database Design	22-23

	Description	Page No.
5.3	API Design	23-24
5.4	UML Diagrams	25-32
5.4.1	Class Diagram	26
5.4.2	Use Case Diagram	26-27
5.4.3	Sequence Diagram	27-28
5.4.4	Activity Diagram	28-29
5.4.5	Component Diagram	29-30
5.4.6	Deployment Diagram	30-31
5.4.7	Collaboration Diagram	31-32
6	MACHINE LEARNING PIPELINE	33-40
6.1	Dataset Analysis	33-35
6.2	Model Architecture	35-36
6.3	Training Configuration	36-38
6.4	Threshold Optimization	38-39
6.5	Model Evaluation and Results	39-40
7	SYSTEM IMPLEMENTATION	41-47
7.1	Backend Implementation	41-44
7.2	Frontend Implementation	45-46
7.3	Security Implementation	46-47
8	SYSTEM STUDY AND TESTING	48-56
8.1	Feasibility Study	48-49
8.2	Software Testing	50-56
9	SOURCE CODE	57-62
10	SCREEN LAYOUTS	63-69
11	FUTURE ENHANCEMENTS	70

	Description	Page No.
12	CONCLUSION	71
13	BIBLIOGRAPHY	72-73

LIST OF FIGURES

Fig. No.	Figure Name	Page No.
5.1	Three-Tier System Architecture Diagram	20
5.4.1	UML Class Diagram	26
5.4.2	UML Use Case Diagram	27
5.4.3	UML Sequence Diagram — Text Analysis Flow	28
5.4.4	UML Activity Diagram — Journaling Workflow	29
5.4.5	UML Component Diagram	30
5.4.6	UML Deployment Diagram	31
5.4.7	UML Collaboration Diagram — Scheduling Workflow	32
10.1	Login / Signup Authentication Interface	63
10.2	Main Journaling Interface with Voice Recording	64
10.3	Emotion Analysis Result with Wellness Suggestions	66
10.4	90-Day Emotion Distribution Bar Chart	67
10.5.1	Account Settings Model	68
10.5.2	Account Settings Model	69

LIST OF TABLES

Table No.	Table Name	Page No.
4.1	Software Requirements Specification	14
4.2	Hardware Requirements Specification	14-15
5.1	User Table Schema	22
5.2	Journal Entry Table Schema	23
5.3	REST API Endpoint Summary	23-24
6.1	GoEmotions 28-Class Label Distribution	33-34
6.3	Model Training Hyperparameters	36-37
6.4	Threshold Sweep Results (Selected)	39
8.2.1	Unit Testing Results	50-51
8.2.2	Integration Testing Results	52-53
8.2.3	System Testing Results	53-54
8.2.4	Acceptance Testing Results	54-55

ABBREVIATIONS

Abbreviation	Full Form
IERRS	Intelligent Emotion Recognition and Recommendation System
AI	Artificial Intelligence
ML	Machine Learning
DL	Deep Learning
NLP	Natural Language Processing
RoBERTa	Robustly Optimized BERT Pretraining Approach
BERT	Bidirectional Encoder Representations from Transformers
BCE	Binary Cross-Entropy
F1	F1 Score (Harmonic Mean of Precision and Recall)
MLP	Multi-Layer Perceptron
CNN	Convolutional Neural Network
RNN	Recurrent Neural Network
LSTM	Long Short-Term Memory
ASR	Automatic Speech Recognition
NLU	Natural Language Understanding
SPA	Single-Page Application
JWT	JSON Web Token
API	Application Programming Interface
REST	Representational State Transfer
HTTP	HyperText Transfer Protocol
CORS	Cross-Origin Resource Sharing
ORM	Object-Relational Mapping
SQL	Structured Query Language
JSON	JavaScript Object Notation
ASGI	Asynchronous Server Gateway Interface

Abbreviation	Full Form
WSGI	Web Server Gateway Interface
UI	User Interface
UX	User Experience
AES-GCM	Advanced Encryption Standard - Galois/Counter Mode
E2EE	End-to-End Encryption
SMTP	Simple Mail Transfer Protocol
OAuth	Open Authorization
UML	Unified Modeling Language
ER	Entity-Relationship
CRUD	Create, Read, Update, Delete
CPU	Central Processing Unit
GPU	Graphics Processing Unit
RAM	Random Access Memory
SSD	Solid State Drive
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JSX	JavaScript XML
CRA	Create React App
HOD	Head of Department
AICTE	All India Council for Technical Education
NBA	National Board of Accreditation
NAAC	National Assessment and Accreditation Council

Chapter 1

INTRODUCTION

Mental health and emotional well-being have emerged as critical concerns in today's fast-paced, high-pressure world where individuals increasingly struggle to track and manage their emotional patterns. Traditional approaches to emotional self-awareness—such as manual journaling, self-reported mood tracking, or sporadic therapy sessions—are often inconsistent, subjective, and insufficient for real-time intervention. These limitations contribute to rising stress, anxiety, burnout, and mental health challenges, particularly among students and professionals facing academic and career pressures. With rapid advancements in Artificial Intelligence (AI), Natural Language Processing (NLP), and speech recognition technologies, there exists tremendous potential to create intelligent systems that automatically detect, analyze, and respond to human emotions with precision and immediacy.

This project presents an **Intelligent Emotion Recognition and Reminder System** through Speech and Text Data, designed to revolutionize personal emotional wellness tracking. The system leverages state-of-the-art transformer-based NLP models like DistilBERT for accurate emotion classification from multimodal inputs analyzing both typed journal entries and spoken voice recordings to detect nuanced emotional states such as joy, sadness, anger, fear, surprise, and neutral. Going beyond mere detection, the system incorporates behavioral psychology principles by automatically identifying prolonged negative emotion patterns and seamlessly integrating with Google Calendar API via OAuth 2.0 to schedule evidence-based wellness interventions including cognitive behavioral therapy (CBT) exercises, mindfulness activities, box breathing techniques, and gratitude journaling prompts.

Built with a modern full-stack architecture featuring **FastAPI** for scalable RESTful backend APIs, **React.js** for responsive web interfaces, and **SQLModel** for secure database management, the system delivers five streamlined modules: User Interaction & Data Input, Speech Processing & Text Preprocessing, Emotion Detection & Analysis, Recommendation & Reminder, and Data Storage & Visualization. Users benefit from real-time emotion feedback, interactive dashboards displaying emotion distribution charts,

Weekly trend line graphs, and calendar heatmaps, all protected by client-side encryption ensuring privacy-first data handling.

This intelligent approach addresses fundamental limitations of existing mental health apps—namely manual effort, shallow sentiment analysis, lack of behavioral integration, privacy vulnerabilities, and absence of long-term pattern visualization. By automating the entire pipeline from multimodal emotion detection through personalized intervention scheduling, the system ensures consistency, scalability, and genuine behavioral impact. Users experience enhanced emotional self-awareness while insurance companies—wait, mental health practitioners and individuals gain from streamlined emotional tracking, reduced dependency on professional intervention for routine wellness, and trustworthy, data-driven insights into emotional patterns.

In summary, the **Intelligent Emotion Recognition and Reminder System** represent a transformative advancement in AI-powered mental health technology. It not only automates emotion detection and psychological intervention but establishes a comprehensive foundation for proactive, privacy-preserving emotional wellness management. This project demonstrates how sophisticated AI tools can be practically deployed through modern web technologies to optimize personal mental health workflows, minimize human subjectivity, and empower better emotional decision-making in everyday scenarios.

Chapter 2

PROJECT DESCRIPTION

2.1 Problem Definition

The traditional approach to emotional wellness monitoring has evolved minimally over the past century. Paper mood diaries, which date back to the early 20th century cognitive behavioral therapy literature, have been directly digitized into smartphone applications without fundamental rethinking of the interaction model or the analytical depth possible with modern AI. This section provides a rigorous technical articulation of the specific problems that IERRS is designed to solve.

The first and most fundamental problem is the semantic poverty of existing emotion taxonomies. Plutchik's wheel of emotions identifies eight primary emotions; Ekman's model recognizes six; most commercial mood-tracking applications use four to ten. While these taxonomies are adequate for pop psychology contexts, they are insufficient for the granularity required by anyone using emotional tracking as a serious wellness or therapeutic practice. The distinction between 'remorse' (characterized by guilt and self-directed negative judgment about a past action) and 'sadness' (characterized by loss and helplessness) has direct therapeutic implications: remorse may be most productively addressed through self-compassion exercises and behavioral amends, while sadness may be better addressed through behavioral activation and social connection. A system that collapses both into 'negative' cannot make this distinction.

The second problem is the computational treatment of journal entries as atomic units. Natural language processing research consistently demonstrates that sentence-level classification outperforms document-level classification for emotion detection tasks because emotional expression is localized at the sentence level. A journal entry of 200 words typically contains 8-12 sentences, and it is entirely normal for consecutive sentences to express opposite emotional valences. Treating the document as a unit requires the model to produce a single probability distribution that averages across all expressed emotions, systematically suppressing emotions expressed in only one or two sentences precisely the kind of nuanced signal that is most clinically interesting.

The third problem is the complete absence of longitudinal awareness in existing tools. From a clinical psychiatry perspective, the temporal dimension of emotional experience is at least as important as its current state. The DSM-5 criteria for major depressive disorder require the presence of depressive symptoms for at least two weeks; bipolar disorder is diagnosed based on cycles of mood elevation and

depression over months or years; persistent depressive disorder (dysthymia) is diagnosed based on a pattern of low mood for at least two years. None of these diagnoses can be made from a single data point. Yet virtually every existing emotional wellness application treats each entry as independent, providing no mechanism to accumulate clinical significance across time.

The fourth problem is the passive nature of existing wellness tools. Research in behavioral economics, particularly the work of Thaler, Sunstein, and Kahneman, demonstrates that the gap between intention and action is a fundamental feature of human psychology, not a character flaw. People consistently fail to follow through on health-promoting behaviors even when they genuinely intend to, due to present bias, status quo bias, and the friction of decision-making under uncertainty. An emotional wellness tool that identifies negative emotional states and presents a suggestion on a screen, leaving it entirely to the user to decide when and how to act on it, is designed to fail. Active scheduling 'I will do a 10-minute box breathing exercise at 6pm today', has been shown to dramatically increase follow-through rates compared to passive intention formation.

The fifth problem is data privacy. Mental health data is among the most sensitive categories of personal information, comparable to medical records, financial data, and biometric identifiers. The major commercial mood-tracking applications like Daylio, Bearable, Moodflow all store user data on cloud servers, typically in plaintext or with server-managed encryption that provides no protection against the application provider's access. This creates a structural misalignment between user interest (privacy of sensitive mental health information) and provider interest (data access for analytics, monetization, and service improvement). A genuinely privacy-first approach requires that the server never receives plaintext journal content, achievable through client-side encryption before transmission.

2.2 Project Details

2.2.1 Core Emotion Detection Engine

The emotion detection engine is the technical centerpiece of IERRS. It is built on RoBERTa-base (roberta-base from Hugging Face), fine-tuned for multi-label sequence classification on the GoEmotions simplified dataset. The choice of RoBERTa over other available architectures (DistilBERT, ALBERT, DeBERTa) represents a deliberate engineering trade-off: RoBERTa provides approximately 5-8% improvement in F1 score over DistilBERT at approximately 2x inference time. For a journaling application where the user submits one entry and waits a few seconds for results, this

latency trade-off is acceptable, and the accuracy improvement justifies the additional computational cost.

The model's input representation uses RoBERTa's Byte-Pair Encoding (BPE) tokenizer with a vocabulary of 50,265 tokens. Inputs are truncated to a maximum length of 128 tokens, which is sufficient for the sentence-level chunks used in the analysis pipeline (the average GoEmotions sample is 12 tokens; sentences in journal entries are typically 15-30 tokens). The model architecture adds a multi-label classification head consisting of a dropout layer ($p=0.1$) and a linear layer mapping RoBERTa's 768-dimensional CLS token representation to 28 output logits, one per emotion class. Sigmoid activation converts logits to independent probabilities for each class, and labels are assigned based on the per-class threshold determined during validation-set sweep.

The training data pipeline converts the native GoEmotions multi-label format (list of class indices per sample) to a 28-dimensional multi-hot vector for each example. A custom `to_multihot()` mapping function creates vectors where position i is 1.0 if class i is positive and 0.0 otherwise. The dataset is tokenized in batches using the `map()` function with `batched=True` for efficiency, and formatted as PyTorch tensors with `dataset.set_format(type='torch')`. The training, validation, and test splits (43,410 / 5,426 / 5,427 examples respectively in the simplified version) are used as provided by the dataset creators.

2.2.2 Weighted Loss Function and Class Imbalance

Class imbalance in the GoEmotions dataset is severe and systematic. The neutral class appears in 29.6% of training examples, while grief appears in only 0.9%. In a naively trained model without imbalance correction, the model learns to predict neutral and admiration reliably while essentially ignoring grief and embarrassment, because the expected loss gradient from these rare classes is negligible relative to the dominant classes. This is precisely the wrong behavior for a clinical application where rare emotions may be the most important signal.

The weighted BCE approach addresses this by computing per-class positive weights using the standard formula derived from the class prior: $w_i = (N - \text{pos}_i) / \text{pos}_i$, where N is the total number of training examples and pos_i is the number of examples where class i is positive. This weight is then passed to PyTorch's `BCEWithLogitsLoss` as the `pos_weight` argument. Intuitively, a weight of 15 for grief means that a false negative on a grief prediction costs 15 times more than a false positive, pushing the

model to prioritize recall for this class. The clipping to [1.0, 15.0] prevents extreme weights for near-zero classes that could destabilize training while ensuring minimum penalization for even the most common class.

2.2.3 Multi-Sentence Chunking Architecture

The `analyze_and_store()` function in `main.py` implements the multi-sentence chunking pipeline. Input text is first stripped of leading and trailing whitespace, then split into sentences using the regex pattern `r'(?<=[!?!]) +'`, which splits on whitespace following sentence-ending punctuation. This approach correctly handles common journal entry structures including single-sentence inputs (treated as a single chunk), multi-sentence paragraphs, and entries ending with exclamation marks or question marks.

Each sentence is independently tokenized and passed through the RoBERTa model within a `torch.no_grad()` context manager to disable gradient computation during inference. The resulting logits are passed through `torch.sigmoid()` to produce per-class probabilities. Emotions are accumulated using two data structures: a Python set (`all_active_emotions`) that accumulates emotion labels whose probability exceeds `EMOTION_THRESHOLD` in any sentence, and a `defaultdict(float)` (`max_scores`) that maintains the maximum observed probability for each emotion across all sentences. The final active emotion list is sorted by descending max score to place the most strongly detected emotion first.

A fallback mechanism handles the edge case where no emotion exceeds the threshold in any sentence (which can occur for very short, emotionally neutral text). In this case, the emotion with the highest probability across all sentences is selected as the sole active emotion, ensuring the response always contains at least one emotion label.

2.2.4 Dynamic Suggestion Engine

The `suggestion_engine.py` module implements a three-layer suggestion generation system. The innermost layer is the emotion cluster mapping, which assigns each of the 28 GoEmotions labels to one of five clusters: `ANXIETY_EMOTIONS` (fear, nervousness), `DEPRESSIVE_EMOTIONS` (sadness, grief, disappointment, remorse), `ANGER_EMOTIONS` (anger, annoyance, disapproval, disgust), `STRESS_EMOTIONS` (confusion, embarrassment), and a positive/neutral catch-all. Each cluster has an associated pool of 10 evidence-based intervention suggestions drawn from cognitive behavioral therapy, mindfulness-based stress reduction, behavioral activation therapy, and somatic experiencing.

The middle layer handles mixed-emotion detection. When a journal entry contains both positive and negative emotions (detected by checking whether any active emotion is in `NEGATIVE_EMOTIONS` and any is not), a separate pool of 5 mixed-emotion suggestions is used, acknowledging the emotional complexity of the entry and providing psychoeducation about emotional coexistence rather than attempting to address only the negative component.

The outer layer implements longitudinal contextualization. The `generate_dynamic_suggestions()` function receives both the current entry's emotions and the user's 90-day history. It computes a negative ratio (total negative emotion occurrences / total emotion occurrences across all history entries) and identifies the dominant historical emotion. Based on the combination of current emotional state and historical pattern, four observation templates are generated: (1) current negative + history negative dominant → persistent pattern acknowledgement with strong action recommendation; (2) current negative + history positive/neutral dominant → acute fluctuation acknowledgement with gentle action recommendation; (3) current positive + history negative dominant → breakthrough acknowledgement with momentum reinforcement; (4) current positive + history positive dominant → stable state acknowledgement with maintenance support.

2.2.5 Scheduling and Notification Pipeline

The scheduling pipeline connects the emotion analysis results to concrete time-bound wellness interventions through two channels. The Google Calendar integration uses the `google-api-python-client` library with OAuth 2.0 authentication (InstalledAppFlow for initial authorization, Credentials persistence in `auth/token.json` for subsequent calls). The `create_calendar_event()` function creates a 30-minute calendar event on the user's primary calendar with the scheduled start time, a wellness-focused title, and the suggestion text in the description field. The event appears immediately in the user's Google Calendar across all devices.

The email notification pipeline uses Python's built-in `smtplib` with Gmail SMTP over SSL (port 465). An `EmailMessage` object is constructed with the full formatted suggestion (Observation, Insights, and Actionable Step sections) in a professional template. APScheduler's `DateTrigger` is used to schedule the `send_email()` function to execute at the exact scheduled datetime, ensuring the reminder arrives precisely when the user has committed to engaging with the wellness activity. The scheduler uses `AsyncIOScheduler` to integrate cleanly with FastAPI's async event loop, and scheduler jobs are given unique IDs with `replace_existing=True` to prevent duplicate job accumulation across server restarts.

2.3 Scope of the Project

The scope of IERRS is defined in three dimensions: target users, technical boundaries, and explicit exclusions.

Target Users: The primary target user is an individual aged 18-35 who practices or wishes to practice regular journaling as a personal wellness habit, has sufficient emotional vocabulary to write or speak about their emotional state in English, and is comfortable using a web application on a desktop or laptop browser. Secondary target users include psychology students and mental health researchers interested in the practical implementation of emotion NLP, and software engineering students interested in full-stack AI application development.

Technical Scope: The system delivers a complete, deployable full-stack application encompassing a trained RoBERTa model with documented training notebook, a FastAPI backend with 18 REST endpoints, a React.js frontend SPA, SQLite data persistence with SQLAlchemy ORM, Google Calendar API integration, Gmail SMTP integration, APScheduler background tasks, multi-format report generation (CSV and Excel), client-side AES-GCM encryption, server-side Fernet encryption, JWT authentication, and bcrypt password security. The system supports both text and voice input, provides 90-day longitudinal analysis, and delivers suggestions through three channels: in-app display, Google Calendar events, and email notifications.

Explicit Exclusions: IERRS explicitly does not provide clinical diagnosis, therapeutic prescription, or crisis intervention. It is not a replacement for professional mental health care, psychotherapy, or psychiatric medication management. It does not support real-time communication with mental health professionals, and it does not claim diagnostic validity for any DSM or ICD condition. Users in acute mental health crisis should be directed to professional crisis services.

Language Scope: The current implementation supports English-language journal entries only, as the GoEmotions training dataset and RoBERTa pre-training corpus are English-language. Extension to other languages via XLM-RoBERTa is identified as a future enhancement.

Deployment Scope: The system is designed for single-user or small-group personal deployment, running on a local machine or a single cloud VM instance.

The SQLite database is appropriate for this scale. Production multi-tenant deployment would require migration to PostgreSQL and additional infrastructure components (load balancer, database replica, distributed APScheduler).

Chapter 3

SYSTEM ANALYSIS

3.1 Existing System

Current emotional wellness tracking systems used by individuals and mental health apps predominantly rely on manual self-reporting methods, which involve users manually selecting mood states, typing journal entries, or rating emotions based on subjective feelings or static scales. Although this process is well-established, it is inherently time-consuming, prone to cognitive biases, and lacks consistency across different users and emotional contexts. These limitations often lead to incomplete emotional awareness, delayed interventions, and ineffective mental health management.

Some semi-automated mental health apps exist, utilizing basic sentiment analysis techniques to identify positive/negative tones in text. However, they still require user supervision and are limited in their ability to accurately classify nuanced emotions or provide actionable interventions. Furthermore, these systems often fail under varying expression conditions such as ambiguous phrasing, cultural differences, sarcasm, or speech with background noise and emotional tone variations.

In many cases, there is no integration of psychological intervention recommendations with behavioral psychology research or automated scheduling capabilities, resulting in generic motivational quotes or ineffective suggestions. The absence of a centralized, AI-driven approach prevents scalability and restricts real-time emotional pattern recognition. Users still rely heavily on self-interpretation, which is inconsistent and difficult to standardize.

3.1.1 Disadvantages

- **Manual Effort Required:**
Users must manually categorize emotions without intelligent analysis support.
- **Inconsistent Self-Assessment:**
Subjective mood interpretation leads to unreliable emotional tracking and disputes with personal reality.
- **Human Cognitive Bias:**
High dependency on users' self-awareness and memory accuracy.
- **No Real-Time Intervention:**
Inability to provide instant emotional feedback or automated wellness responses.
- **No Behavioral Integration:**
Lack of integration with scheduling tools or psychological intervention
- **Poor Scalability:**
Not adaptable for long-term pattern analysis or multi-user deployment.
- **Weak Emotion Processing:**
Basic sentiment analysis lacks precision for nuanced emotions and contextual understanding.

3.2 Proposed System

- IERRS proposes a fundamentally different architecture for emotional wellness monitoring, built on the principle that an effective emotional wellness tool must understand emotional language with clinical nuance, contextualize observations longitudinally, and actively drive users toward evidence-based interventions. The following describes each major component of the proposed system and its role in addressing the deficiencies of existing approaches.

3.2.1 DistilBERT-Based Multi-Label Emotion Detection

Rather than hand-crafted emotion categories or simple sentiment polarity, IERRS uses DistilBERT (distilbert-base-uncased), a knowledge-distilled transformer encoder fine-tuned specifically for the 28-class GoEmotions taxonomy. DistilBERT was selected over the larger RoBERTa-base model which achieved a marginally higher Micro F1 of 0.658 compared to DistilBERT's 0.627 because its approximately 66 million parameters (versus RoBERTa's 125 million) deliver a significantly lower inference latency of 150–300 ms per text entry, which is a practical requirement for a real-time journaling application. This approach provides several advantages:

1. contextual understanding of emotional language that static keyword matching or bag-of-words approaches cannot achieve.
2. The ability to detect emotions even when expressed through metaphor, understatement, or complex syntactic structures.
3. simultaneous multi-label output allowing detection of co-occurring emotions.
4. And a principled training methodology using per-class positive weighting to ensure minority emotion detection.

3.2.2 Sentence-Level Multi-Sentence Aggregation

The proposed system's sentence-level chunking architecture directly addresses the semantic homogenization problem of document-level classification. By analyzing each sentence independently and aggregating through set union and max-score operations, the system correctly represents the full emotional content of a journal entry, including emotions expressed in only a single sentence within a longer entry. This is not a technical nicety but a clinical requirement: the sentence "I had a terrible panic attack this morning" represents clinically important information that must not be diluted by surrounding positive or neutral content.

3.2.3 Longitudinal 90-Day Baseline Analysis

The 90-day analysis window was selected based on clinical literature suggesting that patterns of emotional experience over 3 months are sufficient to identify persistent mood disorders while remaining recent enough to reflect the current life context. The suggestion engine computes the negative emotion ratio and dominant historical emotion across all entries in this window, enabling contextually appropriate characterization of whether today's emotional state represents a sustained pattern or an acute deviation. This temporal contextualization fundamentally changes the nature of the feedback provided to users: from "you feel sad today" to "you feel sad today, which is consistent with the elevated negative mood you have experienced for most of the past three months."

3.2.4 Evidence-Based Clustered Intervention Suggestions

The suggestion engine's intervention recommendations are organized by emotion cluster (anxiety, depression, anger, stress, positive/neutral) and draw from four evidence-based therapeutic modalities: Cognitive Behavioral Therapy (CBT) techniques including cognitive reframing, thought records, and behavioral activation; Mindfulness-Based Stress Reduction (MBSR) techniques including body scan, mindful movement, and sensory grounding; Somatic Experiencing techniques including physiological sigh, cold water exposure response, and progressive muscle relaxation; and Behavioral interventions including physical exercise, social connection, creative activity, and structured task completion. Each modality has strong evidence support from randomized controlled trials, and the assignment of modalities to emotion clusters is guided by established clinical recommendations.

3.2.5 Integrated Scheduling and Delivery

The 30-second auto-schedule countdown represents a deliberate behavioral design choice. When negative emotions are detected, immediately presenting the user with a countdown to auto-scheduling an activity rather than passively displaying suggestion leverages implementation intention research. The user who sees "Auto-scheduling your wellness activity in 30s..." must actively decide whether to cancel, choose a custom time, or allow the default. All three options result in some form of engagement with the suggestion; only active cancellation results in no action. The subsequent Calendar event and email reminder create accountability structures that significantly increase follow-through probability.

3.2.6 Advantages of the Proposed System

- **Clinically Meaningful Emotional Granularity:** 28-class multi-label emotion classification via fine-tuned DistilBERT exceeds all existing consumer applications by a factor of 3–6x in emotional granularity.
- **Sentence-Level Analysis:** Sentence-level chunking with set union aggregation correctly captures mixed emotional states within single journal entries, preserving emotionally significant minority-sentence content.
- **Longitudinal Awareness:** 90-day historical baseline analysis provides temporal context that distinguishes acute fluctuations from persistent patterns, enabling more clinically appropriate response calibration.
- **Evidence-Based Personalized Interventions:** Intervention suggestions drawn from CBT, MBSR, somatic experiencing, and behavioral therapy are assigned to emotion-cluster-specific pools, ensuring recommendations are appropriate to the specific emotional state.
- **Multi-Modal Input (Text and Voice):** Voice journaling via local Whisper ASR removes typing friction and maintains privacy by keeping audio processing entirely on-device.
- **Automated Wellness Action Pipeline:** 30-second auto-schedule countdown, Google Calendar event creation, and email delivery at scheduled time implement implementation intention theory for maximum follow-through.

- **Privacy-First Architecture:** AES-GCM client-side encryption before transmission and Fernet server-side encryption at rest ensure journal content is never accessible in plaintext to the application provider.
- **Rich Reporting and Analytics:** Professional Excel reports with color-coded emotion rows and automated weekly/monthly/yearly email reports provide comprehensive data export and review capabilities.
- **Scalable and Responsive Architecture:** FastAPI async backend, React.js SPA, and APScheduler support concurrent users with low latency of 150-300ms per analysis for text entries.
- **Comprehensive Authentication Security:** Complete JWT authentication, bcrypt password hashing, security question recovery, and 2-step password change flows to implement security best practices throughout the application.

Chapter 4

ENVIRONMENTS

4.1 Software Requirements

Component	Specification	Version
Operating System	Windows 11 / Linux / macOS	—
Backend Language	Python	3.10+
Frontend Language	JavaScript (ES2020+) with JSX via React	Node.js 18+
Web Framework (Backend)	FastAPI (ASGI, async)	0.100+
Frontend Framework	React.js (Create React App)	18.x
Database	SQLite via SQLAlchemy ORM and SQLAlchemy	SQLite 3.x
ML Framework	PyTorch	2.0+
Transformer Library	Hugging Face Transformers	4.35+
ASR Model	OpenAI Whisper (tiny)	20231117
ASGI Server	Uvicorn	0.23+
Version Control	Git + GitHub	Git 2.x
IDE	Visual Studio Code	1.85+
Model Training	Google Colab (T4 GPU)	—
Package Manager (Python)	pip	23+
Package Manager (Node)	npm	9+

4.2 Hardware Requirements

Component	Minimum	Recommended
Processor	Intel Core i5 8th Gen / AMD Ryzen 5 3600	Intel Core i7 12th Gen / AMD Ryzen 7 5800X
RAM	8 GB	16 GB (for model in-memory loading)
Storage	10 GB free disk space	20 GB SSD (for model files and database)

Component	Minimum	Recommended
GPU (Training)	NVIDIA T4 (Google Colab)	NVIDIA RTX 3080 or better
GPU (Inference)	Not required (CPU inference)	Any CUDA-capable GPU
Display	1366 x 768 minimum	1920 x 1080 (Full HD)
Internet	Required for Google APIs and model download	Broadband (50 Mbps+)
Browser	Chrome 100+, Firefox 98+, Edge 100+	Latest stable Chrome or Firefox
Microphone	Optional (for voice journaling)	Built-in or USB microphone

4.3 Software Features and Dependencies

The following sections provide detailed descriptions of each major software dependency, covering its purpose, key features, role in IERRS, and version-specific considerations.

4.3.1 FastAPI

FastAPI is a modern, high-performance web framework for building APIs with Python based on standard Python type hints. It uses Starlette for web components and Pydantic for data validation, providing type-based validation, serialization, and documentation generation with zero additional code. FastAPI is fully compatible with ASGI (Asynchronous Server Gateway Interface), enabling genuine async/await concurrency for I/O-bound operations such as database queries and external API calls.

In IERRS, FastAPI provides the API layer for all 18 endpoints, automatic OpenAPI documentation at /docs, dependency injection for JWT token validation via Depends(get_current_user), Form() and File() parameter types for multipart form data used by the audio analysis endpoint, and middleware for CORS configuration. The framework's native support for async request handlers is critical for the Whisper audio analysis endpoint, which wraps synchronous model inference in an executor to prevent event loop blocking.

Key FastAPI features utilized: dependency injection system for authentication, Path/Query/Form parameter types, HTTPException for standardized error responses, background tasks for non-blocking operations, and the on_event('startup') lifecycle hook for database initialization and scheduler configuration.

4.3.2 SQLAlchemy

SQLModel is a library for interacting with SQL databases from Python code using Python objects. It combines Pydantic's data validation capabilities with SQLAlchemy's database abstraction, providing a unified API that eliminates the impedance mismatch between ORM models and API schemas that

plagues projects using SQLAlchemy and Pydantic independently. SQLModel inherits SQLAlchemy's full feature set including transactions, relationship loading, and complex query construction.

IERRS uses SQLModel for two primary purposes: defining the User and JournalEntry database models using the class User(SQLModel, table=True) syntax, and constructing database queries via the Session context manager with select() statement composition. The JournalEntry.emotion_scores field uses SQLAlchemy's Column(JSON) type to store a dictionary of 28 float values as a JSON column, enabling direct retrieval without deserialization overhead.

SQLAlchemy's create_engine() is configured with SQLite's file-based storage (sqlite:///./journal.db) using default connection pool settings appropriate for SQLite's single-writer concurrency model. For production multi-user deployment, migration to PostgreSQL requires only changing the connection string and installing psycopg2.

4.3.3 PyTorch

PyTorch is an open-source machine learning framework developed by Meta AI Research, providing a dynamic computational graph engine, automatic differentiation (autograd), and deep learning building blocks optimized for GPU acceleration. PyTorch has become the dominant framework for NLP research and is the backend used by Hugging Face Transformers.

In IERRS, PyTorch serves two roles: during model training on Google Colab, it provides the nn.BCEWithLogitsLoss loss function, the Adam optimizer with weight decay, gradient computation and backpropagation, and CUDA GPU acceleration with automatic mixed precision via torch.cuda.amp. During inference in the FastAPI backend, PyTorch provides tensor operations (tokenizer output tensors, sigmoid activation, squeeze and tolist conversion), and the torch.no_grad() context manager for disabling gradient tracking during inference to reduce memory usage and increase throughput.

The torch.sigmoid() function applied to the model's logit outputs produces independent probabilities for each of the 28 emotion classes, which is the correct probability interpretation for multi-label (as opposed to multi-class) classification. In multi-class classification, softmax ensures probabilities sum to 1; in multi-label classification, each class is an independent binary decision, requiring independent sigmoid activations.

4.3.4 Hugging Face Transformers

The Transformers library by Hugging Face provides a unified API for over 200,000 pre-trained models spanning NLP, computer vision, audio, and multimodal tasks. For IERRS, it provides four critical components: AutoTokenizer for loading the DistilBERT WordPiece tokenizer with its 30,522-token vocabulary; AutoModelForSequenceClassification for loading the multi-label classification architecture; the Trainer class with its comprehensive training loop, evaluation, early stopping, and model saving functionality; and the Training Arguments dataclass for configuring all aspects of the training run.

The AutoModelForSequenceClassification class, configured with problem_type='multi_label_classification', automatically adds the appropriate classification head and uses BCEWithLogitsLoss internally during evaluation. The custom WeightedBCETrainer overrides

the `compute_loss()` method to substitute the weighted loss function while preserving all other Trainer functionality including gradient accumulation, mixed precision, evaluation, and model checkpointing.

The `EarlyStoppingCallback`, configured with `early_stopping_patience=3` and monitoring validation micro F1, prevents overfitting by halting training when the validation metric fails to improve for three consecutive epochs and restoring the best checkpoint via `load_best_model_at_end=True`.

4.3.5 OpenAI Whisper

Whisper is an automatic speech recognition system developed by OpenAI, released as open-source software in September 2022. It is trained on 680,000 hours of multilingual and multitask supervised data collected from the internet, producing a model that is both highly accurate and remarkably robust to accents, noise, technical vocabulary, and code-switching. Unlike cloud-based ASR services such as Google Cloud Speech-to-Text or Amazon Transcribe, Whisper runs entirely locally, ensuring that voice journal content never leaves the user's machine.

IERRS uses the tiny model variant (39 million parameters, approximately 150 MB on disk) as an optimal trade-off between transcription speed (approximately 0.5-2 seconds per 30-second audio clip on CPU) and accuracy (word error rate of approximately 5-10% for clear English speech). The model accepts audio files in WebM format, the native output format of the browser's MediaRecorder API, and returns a dictionary containing the transcribed text, detected language, and word-level timestamps.

A critical implementation detail is the handling of Whisper's synchronous Python API within FastAPI's async framework. Since Whisper's `transcribe()` method is blocking it runs a CPU-intensive inference loop without yielding control calling it directly from an async FastAPI endpoint would block the entire event loop, preventing any other requests from being processed during transcription. The correct pattern is to wrap the call in `asyncio.get_event_loop().run_in_executor(None, partial(whisper_model.transcribe, path))`, which executes the blocking call in a thread pool executor without blocking the event loop.

4.3.6 Passlib and python-jose

Passlib is a password hashing library for Python providing implementations of 30+ hashing algorithms. IERRS uses `CryptContext` with the `bcrypt` scheme and `deprecated='auto'`, which selects `bcrypt` as the primary scheme and automatically re-hashes passwords using deprecated schemes if encountered. `Bcrypt` is the industry standard for password hashing due to its adaptive cost factor: as hardware becomes faster, the cost factor can be increased to maintain adequate hashing time, preventing brute-force attacks.

`python-jose` is a JOSE (JSON Object Signing and Encryption) implementation providing JWT creation and validation. IERRS uses the HS256 (HMAC with SHA-256) algorithm for token signing, a 24-hour token expiry, and the user's username as the 'sub' claim. JWT validation in the `get_current_user()` dependency verifies the token signature using the `SECRET_KEY`, extracts the username, and queries the database to confirm the user exists. This stateless authentication pattern requires no server-side session storage.

4.3.7 APScheduler

APScheduler (Advanced Python Scheduler) is a task scheduling library providing cron, interval, and date-based triggers for Python applications. It supports multiple job stores (memory, SQLAlchemy, MongoDB, Redis) and executors (thread pool, process pool, asyncio). IERRS uses AsyncIOScheduler with in-memory job storage and asyncio executor, appropriate for a single-process deployment.

Three types of jobs are configured:

1. Date Trigger jobs scheduled via `scheduler.add_job(send_email, 'date', run_date=dt)` for per-user wellness reminder email delivery at the user-specified or auto-determined time;
2. CronTrigger weekly report jobs (`day_of_week='sun', hour=9`) for Sunday morning report emails to all users;
3. CronTrigger monthly (`day=1`) and yearly (`month=1, day=1`) report jobs. All jobs are assigned unique IDs and configured with `replace_existing=True` to prevent duplicate job creation across server restarts.

4.3.8 Cryptography (Fernet)

The cryptography library provides the Fernet symmetric encryption scheme, an implementation of AES-128-CBC with HMAC-SHA256 authentication. Fernet guarantees that encrypted messages cannot be manipulated or read without the key. In IERRS, a persistent Fernet key is generated once, stored in the `.env` file as `FERNET_KEY`, and loaded at server startup. Journal content is encrypted using `cipher_suite.encrypt(text.encode())` before storage and decrypted using `cipher_suite.decrypt(encrypted_content).decode()` for retrieval.

A critical security requirement is key persistence across server restarts. The anti-pattern of calling `Fernet.generate_key()` at startup generates a new random key on each restart, making all previously encrypted content permanently unreadable. The corrected implementation loads the key from the `FERNET_KEY` environment variable, raising `RuntimeError` if not set, ensuring the key is established once and never changed without a deliberate data migration.

4.3.9 React.js and Supporting Libraries

React.js (version 18) provides the component-based, declarative UI framework for IERRS's frontend. The application uses functional components exclusively with React Hooks `useState` for state management, `useEffect` for lifecycle and side effects, and `useRef` for mutable references to the auto-schedule timer. The single-file `App.js` architecture was chosen for simplicity appropriate to the project scale, with `EmotionBarChart.js` and `ReportDownload.js` extracted as separate components due to their self-contained functionality.

`react-chartjs-2` (Chart.js wrapper for React) provides the `EmotionTimeline` bar chart with a 28-color emotion palette, responsive sizing, custom tooltips, and clean gridline styling. `Lucide-react` provides consistent SVG icon components (Mic, Send, Logout, Calendar, Settings, X, CheckCircle) throughout the interface. The fetch API (native browser, without Axios) handles all API communication with JWT Authorization headers.

The Web Crypto API (`window.crypto.subtle`) provides AES-256-GCM client-side encryption in `encryption.js`. The `getOrCreateKey()` function generates a new 256-bit AES key on first use, exports and stores it in `localStorage` as base64, and imports it on subsequent uses. The `encryptText()` function generates a random 12-byte IV per encryption operation, performs AES-GCM encryption, prepends the IV to the ciphertext, and returns the base64-encoded combined buffer. This ensures that the same plaintext produces different ciphertext on each encryption due to the random IV, preventing pattern-matching analysis.

4.3.10 openpyxl

`openpyxl` is a Python library for reading and writing Excel 2010+ files (.xlsx). It provides Workbook and Worksheet objects, cell formatting via `PatternFill`, `Font`, `Alignment`, and `Border` classes, and programmatic column width configuration. IERRS's `generate_excel_report()` function uses `openpyxl` to produce professional-quality emotion reports with: an indigo header row (HEX 4F46E5) with white bold text, alternating gray row shading (F9FAFB), red cell fill for negative emotion entries (FEE2E2), green cell fill for positive emotion entries (D1FAE5), thin gray borders on all cells (D1D5DB), center alignment for date and emotion columns, left alignment with word wrap for text columns, and optimized column widths (Date: 15, Input: 55, Emotion: 25, Action: 65). The resulting file is returned as base64-encoded bytes for transmission to the browser, which constructs a Blob and triggers a file download.

4.3.11 Google Calendar API and google-auth

The Google Calendar API v3 provides programmatic access to Google Calendar functionality including event creation, listing, updating, and deletion. The `google-api-python-client` library provides the `build('calendar', 'v3', credentials=creds)` service object. OAuth 2.0 authentication is implemented using `InstalledAppFlow`, which opens a browser tab for the user to authorize calendar access on first run, then stores the resulting credentials in `auth/token.json`. On subsequent runs, the credentials are loaded from the file and refreshed automatically when expired using `creds.refresh(Request())`.

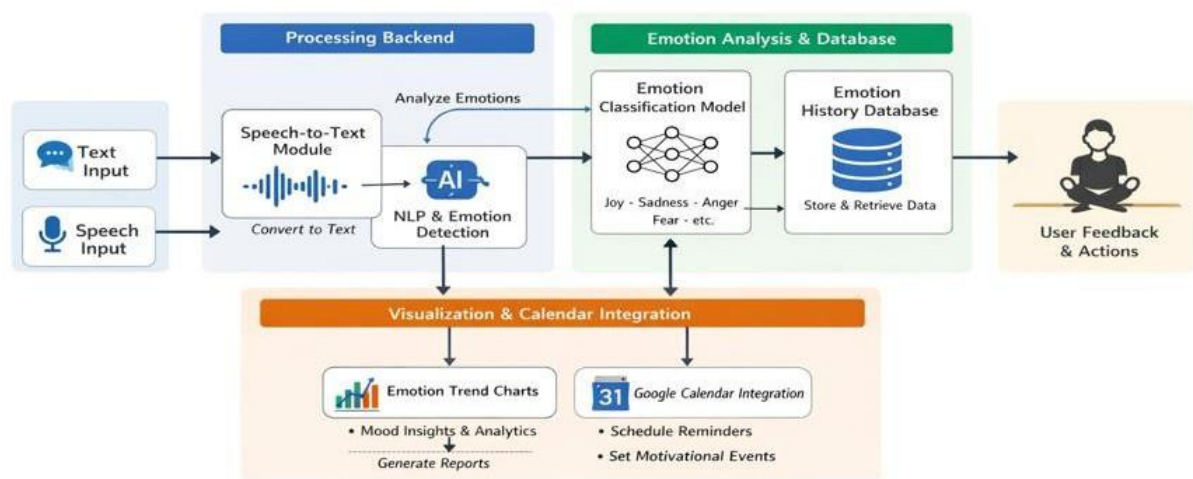
Event creation uses the `events().insert()` method with a calendar event body specifying the event summary, description, start time in ISO 8601 format with Asia/Kolkata time zone, and end time 30 minutes after the start. The event appears immediately in the user's Google Calendar across all devices. The returned event object contains the event ID and an HTML Link for direct browser access to the event.

Chapter 5

SYSTEM DESIGN

5.1 System Architecture

IERRS employs a three-tier client-server architecture with a strict separation between the presentation tier (React.js frontend), the application/business logic tier (FastAPI backend), and the data tier (SQLite database with Fernet-encrypted content). This architecture supports independent development, testing, and scaling of each tier, and follows established software engineering principles of separation of concerns, single responsibility, and dependency inversion.



5.1.1 Presentation Tier

The React.js SPA constitutes the presentation tier, running entirely in the user's browser. Communication with the backend occurs exclusively through HTTP REST API calls using the browser's native fetch API. No server-side rendering is used; the React application is served as static files (HTML, JavaScript, CSS) from any static file host.

5.1.2 Application Tier

The FastAPI backend constitutes the application tier, running as a Uvicorn ASGI process on the host machine.

The application tier is organized into five functional modules: the Authentication Module handling user registration, login, JWT generation, and password management; the Analysis Module implementing the multi-sentence chunking pipeline and emotion classification; the Suggestion Module computing longitudinal baselines and generating structured wellness recommendations; the Scheduling Module managing APScheduler job creation for email delivery and coordinating Google Calendar event creation; and the Reporting Module generating formatted CSV and Excel exports from journal history data.

5.1.3 Data Tier

The SQLite data tier stores persistent application data through two tables managed by SQLAlchemy. User records store authentication credentials and profile preferences. JournalEntry records store the per-entry analysis results including the primary emotion label string, the full 28-class probability scores as a JSON object, the Fernet-encrypted journal content as bytes, and the plaintext input sentence for suggestion generation context. Database operations use SQLAlchemy's Session context manager with explicit commit() calls, ensuring ACID compliance for all write operations.

The data tier design reflects a deliberate trade-off between privacy and functionality. Storing the plaintext input_sentence alongside the encrypted_content enables the suggestion engine to access the raw text for pattern analysis and suggestion generation without requiring a decryption round-trip. In a full privacy-first implementation, all suggestion generation would use only the stored emotion labels rather than raw text, eliminating the plaintext column. This architecture evolution is identified as a future security enhancement.

5.1.4 Cross-Cutting Architecture Concerns

Security: Authentication uses JWT tokens validated at every protected endpoint via the Depends(get_current_user) dependency injection pattern. Passwords are hashed with bcrypt using CryptContext. Journal content is encrypted server-side with Fernet and client-side with AES-GCM. The SECRET_KEY and FERNET_KEY are loaded from environment variables, raising RuntimeError at startup if not set.

Error Handling: All endpoint functions wrap database operations and model inference in try-except blocks, raising HTTPException with appropriate status codes (400 for validation errors, 401 for

authentication failures, 500 for unexpected errors). Date parsing from user input uses `strptime()` within a try-except that raises `HTTPException(400, 'Invalid date format')` for malformed inputs.

CORS Configuration: The `CORSMiddleware` is configured to allow requests from server with all methods, all headers, and credentials. For production deployment, the `allow_origins` list should be updated to the production frontend domain.

Startup Configuration: The `on_event('startup')` handler creates database tables via `SQLModel.metadata.create_all(engine)`, starts the `APScheduler`, and registers cron jobs with unique IDs and `replace_existing=True` to prevent duplicate job accumulation.

5.2 Database Design

The database schema consists of two tables defined as `SQLModel` classes in `models.py`. The following describes each table's columns, data types, constraints, and relationships.

5.2.1 User Table

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique user identifier
username	VARCHAR	UNIQUE, INDEX, NOT NULL	Unique username for login
email	VARCHAR	UNIQUE, INDEX, NOT NULL	User email address
hashed_password	VARCHAR	NOT NULL	Bcrypt hash of user password
default_morning_time	VARCHAR	DEFAULT '06:00'	Default morning scheduling time (HH:MM)
default_evening_time	VARCHAR	DEFAULT '17:00'	Default evening scheduling time (HH:MM)
security_question	VARCHAR	NULLABLE	Security question for password recovery
hashed_security_answer	VARCHAR	NULLABLE	Bcrypt hash of security answer (lowercased)

5.2.2 JournalEntry Table

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique entry identifier
user_id	INTEGER	FOREIGN KEY(user.id), NOT NULL	Reference to owning User record
date	DATE	NOT NULL	Journal entry date (YYYY-MM-DD)
input_sentence	TEXT	NOT NULL	Raw plaintext journal entry
encrypted_content	BLOB	NOT NULL	Fernet-encrypted journal content
emotion_primary	VARCHAR	NOT NULL	Comma-separated active emotion labels
emotion_scores	JSON	NOT NULL	Dict[str, float] of 28-class probabilities

5.2.3 Database Relationships and Indexes

The User table has a one-to-many relationship with JournalEntry via the user_id foreign key. SQLAlchemy creates a database index on User.username and User.email for $O(\log n)$ lookup performance during authentication (login queries user by username or email). Compound queries in the suggestion engine and timeline endpoint filter by both user_id and date range, which SQLite optimizes using the rowid index on the INTEGER primary key combined with linear scan on the date column. For high-volume deployments, a composite index on (user_id, date) would improve query performance for the 90-day history retrieval.

5.3 API Design

The IERRS REST API follows RESTful design principles with resource-based URL structure, appropriate HTTP method usage, and consistent JSON response formats. All protected endpoints require a valid JWT Bearer token in the Authorization header. The following table summarizes all 18 API endpoints.

Method	Endpoint	Auth	Description
POST	/signup	No	Create new user account
POST	/token	No	Authenticate and receive JWT token
POST	/forgot-password/question	No	Retrieve security question for reset

Method	Endpoint	Auth	Description
POST	/forgot-password/reset	No	Reset password via security answer
GET	/profile	Yes	Get current user profile data
PUT	/profile	Yes	Update scheduling time preferences
PUT	/profile/security-question	Yes	Set/update security question and answer
POST	/profile/verify-password	Yes	Verify current password before change
PUT	/profile/password	Yes	Change user password
DELETE	/profile/account	Yes	Delete account and all journal data
POST	/analyze-text	Yes	Submit text entry for emotion analysis
POST	/analyze-audio	Yes	Submit audio blob for transcription and analysis
POST	/schedule-activity	Yes	Create Calendar event and queue email reminder
GET	/timeline	Yes	Get emotion history for charting (default 90 days)
GET	/report/csv	Yes	Export journal history as CSV
GET	/report/excel	Yes	Export journal history as base64-encoded XLSX

5.3.1 Request and Response Formats

The /analyze-text endpoint accepts multipart form data with fields: text (string, required) and date (string, YYYY-MM-DD format, required). It returns a JSON response with: emotions (array of strings, sorted by descending confidence score), suggestion (object with observation, insight, and action string fields), input_text (string, the submitted journal text for display), and requires_scheduling (boolean, true when any active emotion is in NEGATIVE_EMOTIONS).

The /analyze-audio endpoint accepts multipart form data with fields: file (binary, audio blob in WebM format) and date (string). It returns the same structure as /analyze-text with the additional field transcription (string, the Whisper-transcribed text). The /report/excel endpoint returns a JSON object with filename (string, e.g., 'IERRS_monthly_2024-01-01_to_2024-01-31.xlsx') and content (string, base64-encoded XLSX file bytes), which the frontend decodes using atob() and constructs into a downloadable Blob.

5.4 UML Diagrams :

The following sections describe each of the UML diagrams developed for IERRS, covering the system's static structure, behavioral dynamics, and physical deployment topology. The Unified Modeling Language (UML) is a standardized general-purpose modeling language for software systems, providing a shared visual vocabulary for communicating architecture and behavior among developers, stakeholders, and documentation readers.

5.4.1 Class Diagram

The Class Diagram models the static structure of the IERRS system at the class level. The primary domain model classes are User (ORM model with attributes: id, username, email, hashed_password, default_morning_time, default_evening_time, security_question, hashed_security_answer) and JournalEntry (ORM model with attributes: id, user_id, date, input_sentence, encrypted_content, emotion_primary, emotion_scores) connected by a composition relationship. The User class has operations: verify_password(), get_password_hash().

Service classes include EmotionAnalyzer (attributes: model_path, threshold, label_names, tokenizer, model; operations: analyze(text), split_sentences(text), aggregate_scores(sentence_scores)), SuggestionEngine (operations: get_suggestion_tip(emotions), generate_dynamic_suggestions(current, history)), ReportGenerator (operations: generate_csv_report(user_id, range_type, engine), generate_excel_report(user_id, range_type, engine)), SchedulerService (operations: schedule_activity(suggestion, dt), send_email(to, subject, body)), and GoogleCalendarService (attributes: credentials; operations: get_service(), create_event(summary, description, dt)). The EmotionAnalyzer has an aggregation relationship with the FastAPI Application class.

Pydantic schema classes include UserCreate (attributes: username, email, password), ProfileUpdate (attributes: default_morning_time, default_evening_time), PasswordChange (attributes: current_password, new_password), and SecurityQuestionUpdate (attributes: question, answer). These represent dependency relationships from the API endpoint functions to the Pydantic classes.

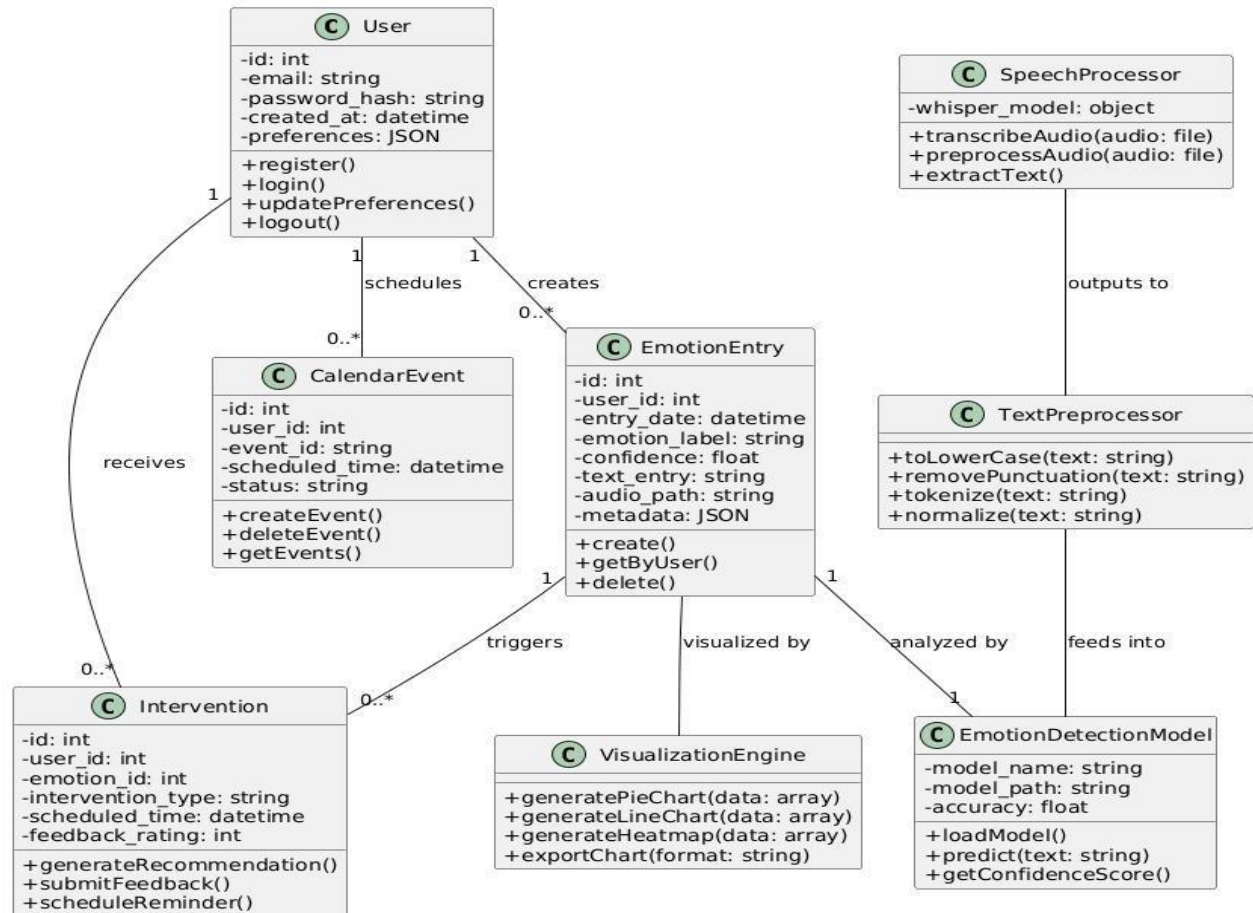


Fig. 5.4.1: UML Class Diagram

5.4.2 Use Case Diagram

The Use Case Diagram identifies two actors: End User (human user interacting with the application) and System Scheduler (automated background process). End User use cases: Register Account (includes Validate Email Format, Hash Password), Login (includes Validate Credentials, Generate JWT), Submit Text Journal Entry (includes Encrypt Text, Analyze Emotions, Generate Suggestion), Submit Voice Journal Entry (includes Record Audio, Transcribe with Whisper, Analyze Emotions),

View Emotion Results (includes Display Emotion Tags, Display Suggestion), Schedule Activity (<<extend>> from View Emotion Results; includes Create Calendar Event, Queue Email Reminder), View Emotion History Chart (includes Fetch 90-Day Timeline), Download Report (includes Generate Excel File, Return Base64), Update Settings (includes Change Password, Set Security Question, Update Schedule Preferences), Delete Account (includes Confirm Password, Cascade Delete Journals). System Scheduler use cases: Send Weekly Report Email, Send Monthly Report Email, Send Yearly Report Email, Deliver Scheduled Wellness Reminder.

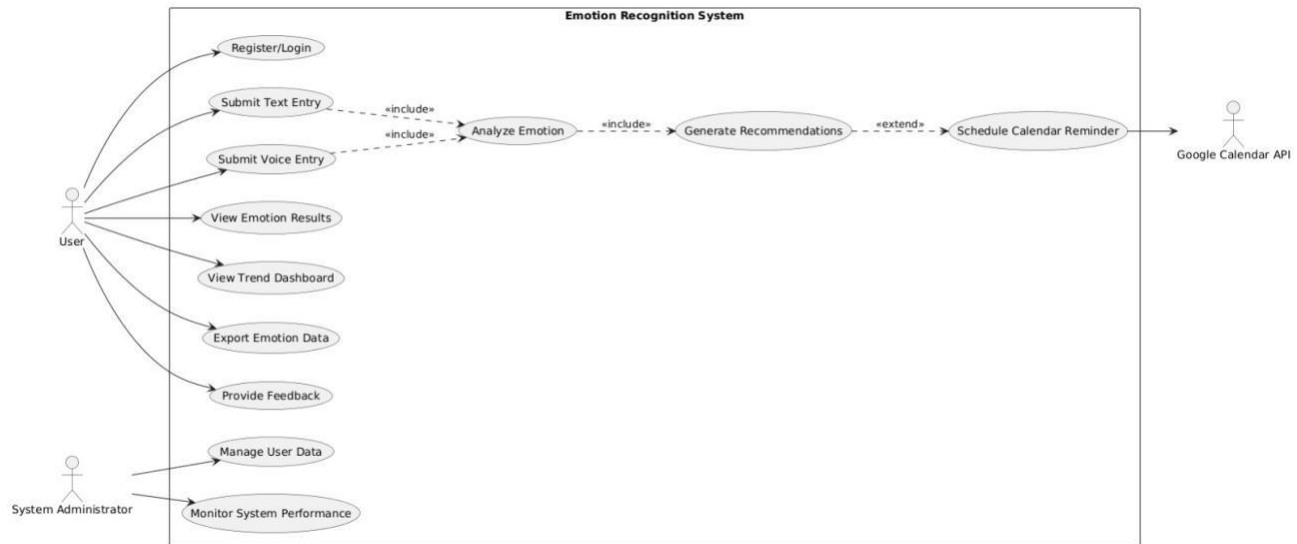


Fig. 5.4.2: UML Use Case Diagram

5.4.3 Sequence Diagram: Text Analysis Flow

The Sequence Diagram for the text analysis workflow shows the time-ordered message exchange between six objects: User, React Frontend, FastAPI Backend, EmotionAnalyzer, SuggestionEngine, and SQLite Database. The sequence proceeds as follows.

User types the text and clicks Analyze. Frontend calls `encryptText(text)` via the Web Crypto API (async), awaits the encrypted string. Frontend sends `POST /analyze-text` with encrypted text and date as form data; the Authorization header contains the JWT. Backend's `get_current_user()` dependency queries SQLite for the user by JWT username. Backend calls `analyze_and_store(text, date, user)`. `analyze_and_store` splits text into sentences. For each sentence, Backend calls `EmotionAnalyzer.analyze(sentence)` returning an `emotion_scores` dictionary. Backend aggregates scores and determines `active_emotions`. Backend calls SQLite `INSERT JournalEntry`. Backend queries SQLite `SELECT JournalEntry WHERE user_id AND date >= 90 days ago`. Backend calls `SuggestionEngine.generate(active_emotions, history)` returning `suggestion_dict`. Backend returns

{emotions, suggestion, requires_scheduling} JSON to Frontend. Frontend renders emotion tags and suggestion panel. If requires_scheduling is true, Frontend starts a 30-second countdown timer.

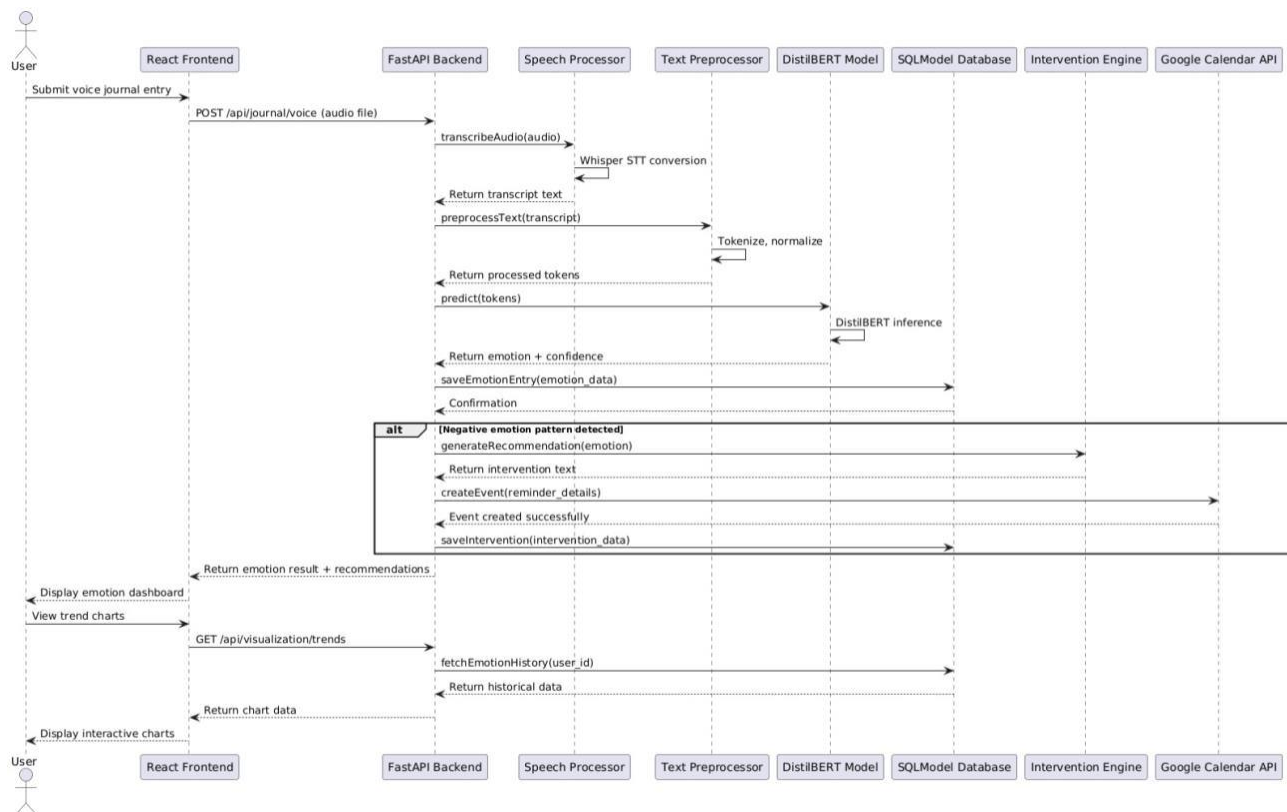


Fig. 5.4.3: UML Sequence Diagram: Text Analysis Flow

5.4.4 Activity Diagram: Journaling Workflow

The Activity Diagram models the complete journaling workflow from application open to post-analysis follow-through. The workflow begins at an initial node and proceeds through: Check Authentication (decision: valid JWT in localStorage? yes → Load Main Interface; no → Show Login Form). Login Flow: Enter credentials → POST /token → decision: valid? yes → Store JWT, Load Main Interface; no → Show error. Main Interface Activity: User chooses Text Entry or Voice Recording.

Text Entry path: Type journal entry → Click Analyze → Analyze button validation (disabled if text empty or isRecording) → POST /analyze-text → Display results. Voice Recording path: Click Record → MediaRecorder starts → Click Stop → MediaRecorder stops, audio blob collected → POST /analyze-audio → Whisper transcription → Display results with transcription.

After results display: decision: requires_scheduling? yes → Start 30s countdown fork: user clicks cancel → countdown cleared; user picks custom time → send custom time to /schedule-activity; countdown reaches 0 → send 'auto' to /schedule-activity. Schedule Activity: Create Calendar event (parallel fork) → Queue email reminder → Show success confirmation. End node.

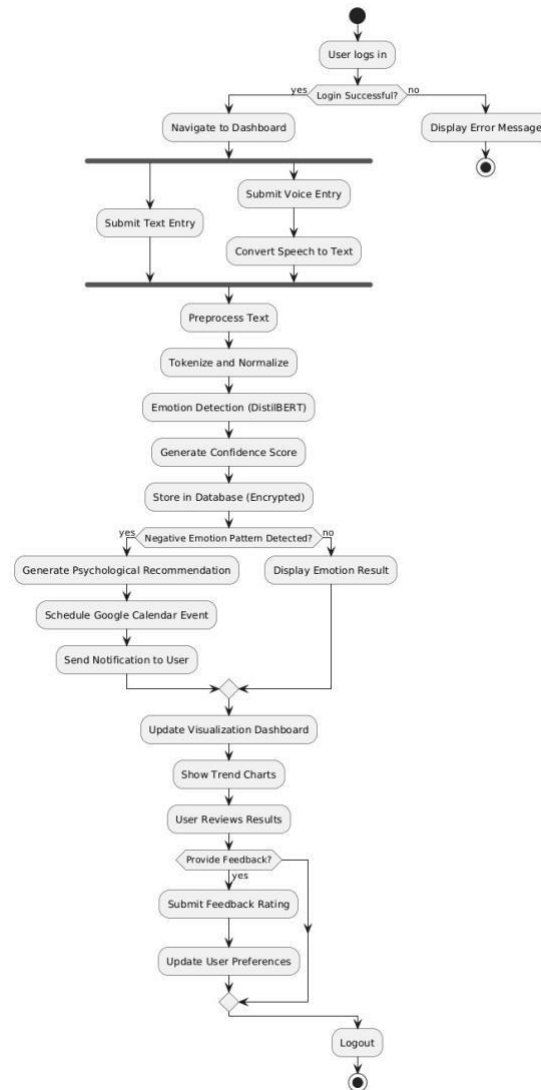


Fig. 5.4.4: UML Activity Diagram: Journaling Workflow

5.4.5 Component Diagram

The Component Diagram illustrates the physical software component structure and inter-component dependencies. Components: React Frontend Component (contains: App.js, EmotionBarChart.js, ReportDownload.js, encryption.js, App.css) provides: User Interface; requires: FastAPI REST API. FastAPI Application Component (contains: main.py, models.py) provides: REST API; requires: ML Model Files, SQLite DB, Google Calendar API, Gmail SMTP, APScheduler. ML Model Component (contains: model.safetensors, tokenizer.json, model_meta.json, tokenizer_config.json) provides: Emotion Classification; required by: FastAPI Application. Services Component (contains: suggestion_engine.py, report_generator.py, emotion_analytics.py) provides: Suggestion Generation,

Report Generation, Analytics; required by: FastAPI Application. Whisper ASR Component (whisper package, tiny model weights) provides: Speech-to-Text Transcription; required by: FastAPI Application. External Google Calendar API Component provides: Calendar Event Management; required by: FastAPI Application. External Gmail SMTP Server provides: Email Delivery; required by: FastAPI Application. SQLite Database Component (journal.db) provides: Data Persistence; required by: FastAPI Application.

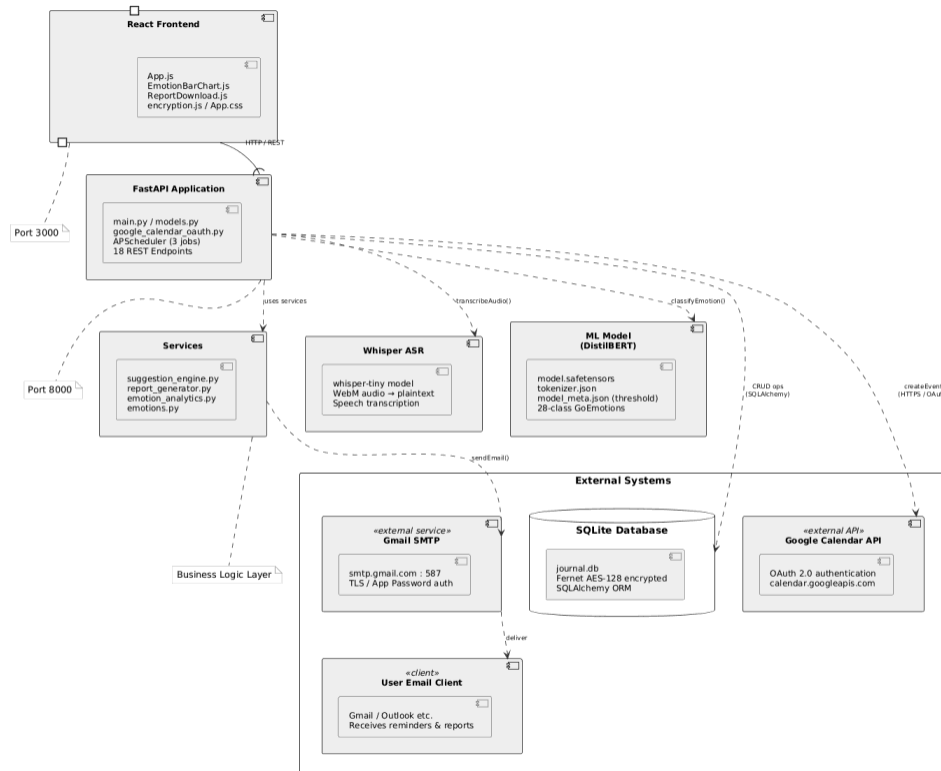


Fig. 5.4.5: UML Component Diagram

5.4.6 Deployment Diagram

The Deployment Diagram describes the physical deployment topology. Three nodes are identified: Developer/User Machine, External Google Services, and User's Email Client. The Developer/User Machine node hosts two artifacts: (1) React Dev Server node (Node.js process, port 3000) executing the React Application artifact (`App.js`, `EmotionBarChart.js`, `ReportDownload.js`, static assets); and (2) FastAPI Server node (Uvicorn process, port 8000) executing the FastAPI Application artifact (`main.py`, `services/`, `constants/`), the DistilBERT Model artifact (`./model/` directory), the SQLite Database artifact (`journal.db`), and the Whisper ASR Model artifact (loaded in memory). Communication between React Dev Server and FastAPI Server occurs over HTTP on localhost. The

FastAPI Server communicates with External Google Services node over HTTPS: Google Calendar API (calendar.googleapis.com) and Gmail SMTP (smtp.gmail.com:587). The User's Email Client node receives emails from Gmail SMTP at the scheduled time.

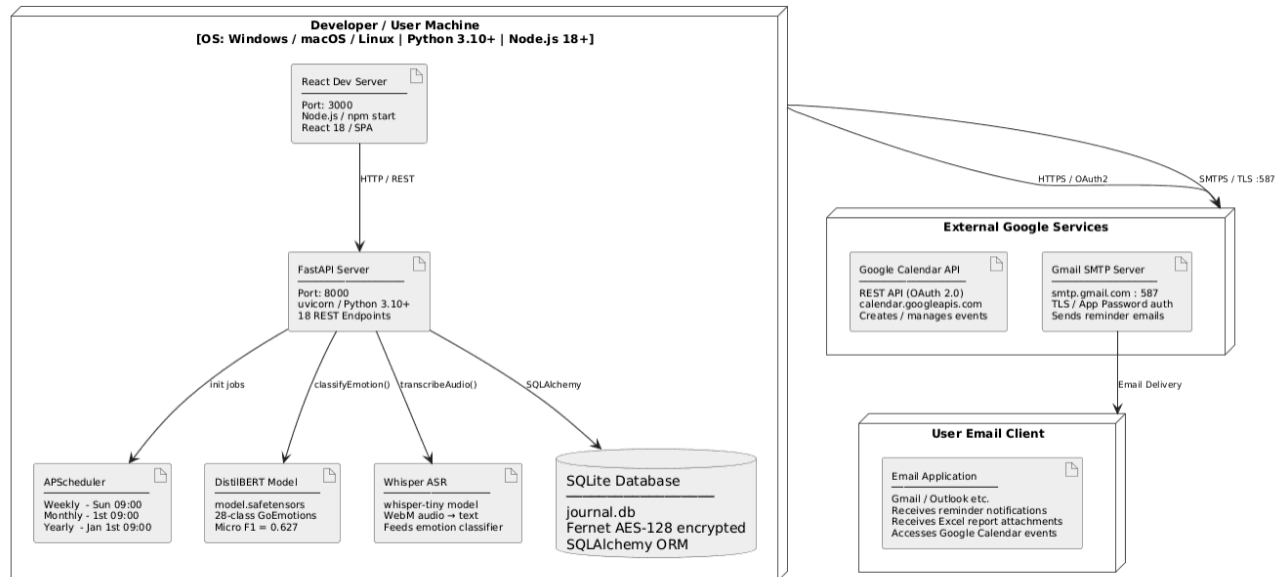


Fig. 5.4.6: UML Deployment Diagram

5.4.7 Collaboration Diagram: Scheduling Workflow

The Collaboration Diagram for the scheduling workflow emphasizes structural relationships alongside message flows. Objects participating: FrontendApp, FastAPIApp, GoogleCalendarService, APScheduler, GmailSMTP, UserEmailClient. Numbered message flows:

1. FrontendApp → FastAPIApp: POST /schedule-activity {suggestion, scheduled_time}
2. FastAPIApp → FastAPIApp: resolve_datetime(scheduled_time) — parse 'auto' or ISO string to datetime object
3. FastAPIApp → GoogleCalendarService: create_event(summary, description, dt) — parameters: summary='IERRS Scheduled Activity', description='Please check your email', start=dt, end=dt+30min
4. GoogleCalendarService → Google Calendar API: events().insert(calendarId='primary', body=event).execute()
5. Google Calendar API → GoogleCalendarService: return {event_id, htmlLink}
6. FastAPIApp → APScheduler: add_job(send_email, 'date', run_date=dt, args=[user.email, subject, body])

7. [At scheduled time] APScheduler → GmailSMTP: send_email(user.email, 'IERRS: Scheduled Wellness Reminder', body)
8. GmailSMTP → UserEmailClient: deliver email with formatted suggestion content
9. FastAPIApp → FrontendApp: return {message: 'Activity scheduled for HH:MM AM/PM on Mon DD'}

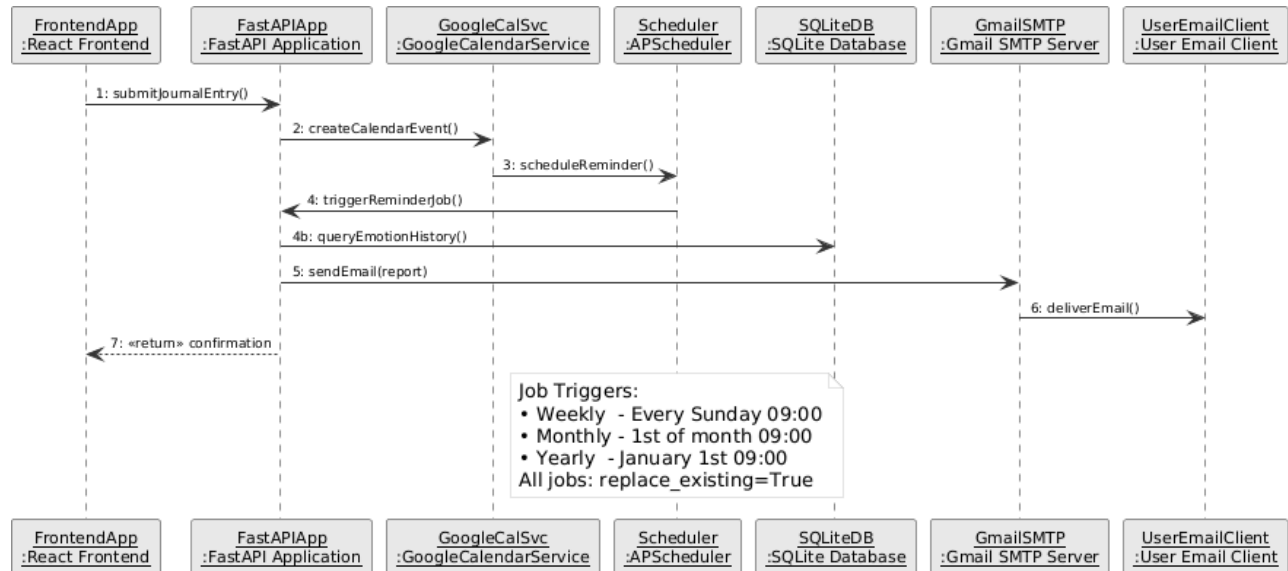


Fig. 5.4.7: UML Collaboration Diagram: Scheduling Workflow

Chapter 6

MACHINE LEARNING PIPELINE

6.1 Dataset Analysis

The GoEmotions dataset (simplified version) is used for training the emotion classification model. It was introduced by Demszky et al. (2020) at Google Research and represents the largest manually annotated fine-grained emotion dataset in English. The dataset consists of 58,009 Reddit comments (from the r/all subreddit between 2005 and 2019) annotated by trained raters across 28 emotion categories. The simplified version consolidates some of the original 27 non-neutral categories into a standard 28-class taxonomy.

Dataset splits: training set (43,410 examples, 74.8%), validation set (5,426 examples, 9.3%), test set (5,427 examples, 9.3%), and approximately 3,000 examples excluded due to annotation disagreement. Each example consists of a text string (average 12.4 tokens, range 1–128 tokens) and a list of zero or more emotion labels (multi-label format). 14.6% of examples have multiple labels, reflecting the genuine multi-label nature of emotional expression. The distribution of label counts per example: 0 labels (0.8%, treated as neutral by the simplified version), 1 label (80.2%), 2 labels (16.3%), 3+ labels (2.7%).

6.1.1 Class Distribution Analysis

The most critical characteristic of the GoEmotions dataset for model training purposes is its severe class imbalance. The following table shows the distribution of all 28 emotion classes in the training set, along with the computed positive weights used in the weighted BCE loss function.

Emotion	Train Count	Frequency (%)	Pos Weight
neutral	18,628	29.6%	1.0
admiration	9,637	15.3%	1.0
approval	6,181	9.8%	1.14
annoyance	5,347	8.5%	1.29
gratitude	4,931	7.8%	1.39
disapproval	4,468	7.1%	1.54

Emotion	Train Count	Frequency (%)	Pos Weight
amusement	4,228	6.7%	1.62
curiosity	3,756	5.9%	1.83
joy	3,344	5.3%	2.05
love	2,989	4.7%	2.30
disappointment	2,808	4.5%	2.45
anger	2,609	4.1%	2.64
sadness	2,524	4.0%	2.73
optimism	2,449	3.9%	2.82
caring	2,179	3.5%	3.16
confusion	1,896	3.0%	3.63
surprise	1,829	2.9%	3.76
fear	1,537	2.4%	4.47
realization	1,400	2.2%	4.91
desire	1,168	1.9%	5.89
excitement	1,072	1.7%	6.42
disgust	962	1.5%	7.14
remorse	787	1.2%	8.73
nervousness	667	1.1%	10.30
relief	573	0.9%	12.0
pride	530	0.8%	15.0 (clipped)
embarrassment	415	0.7%	15.0 (clipped)
grief	389	0.6%	15.0 (clipped)

6.1.2 Text Characteristics

The GoEmotions text samples are Reddit comments characterized by informal register, internet language conventions (abbreviations, acronyms, emoticons), occasional profanity, and highly contextual meaning. This domain-specific language presents challenges for pre-trained models

primarily trained on more formal corpora. DistilBERT, distilled from BERT-base and pre-trained on English Wikipedia and BookCorpus, inherits BERT's WordPiece tokenization and contextual representations. While its pre-training corpus is less diverse than larger models, its knowledge distillation training process preserves approximately 97% of BERT-base's language understanding capability at roughly half the model size, making it well-suited for informal short-text classification tasks such as GoEmotions.

The short average length of GoEmotions samples (12.4 tokens) means that most samples fit well within the 128-token maximum sequence length used in training, with no truncation in the vast majority of cases. Journal sentences in IERRS (typically 15–30 tokens) are slightly longer but still well within the truncation threshold.

6.2 Model Architecture

6.2.1 *DistilBERT-Base Architecture*

DistilBERT-base-uncased is a 6-layer transformer encoder produced by knowledge distillation from BERT-base. It has 12 attention heads per layer, a hidden size of 768 dimensions, an intermediate FFN hidden size of 3,072 dimensions, and a total of approximately 66 million trainable parameters roughly half the size of BERT-base's 110 million and RoBERTa-base's 125 million. The input representation uses a learned embedding matrix of 30,522 tokens (WordPiece vocabulary inherited from BERT), position embeddings up to 512 positions, and token type embeddings inherited from BERT's architecture. The output of the model is a sequence of 768-dimensional vectors, one per input token.

For sequence classification, the [CLS] token's output vector (position 0 in the output sequence) is used as the pooled representation of the entire input. In the Hugging Face `AutoModelForSequenceClassification` implementation for multi-label classification, this CLS representation passes through a dropout layer ($p=0.1$) followed by a linear projection layer mapping 768 dimensions to `num_labels=28` dimensions, producing 28 raw logit values. The sigmoid function then converts each logit independently to a probability in $[0,1]$, reflecting the multi-label nature of the classification where each class is an independent binary prediction.

6.2.2 *Multi-Label vs Multi-Class Classification*

A critical architectural distinction is the difference between multi-label and multi-class classification. Multi-class classification (e.g., digit recognition with 10 classes) assumes mutual exclusivity: exactly one class is correct for each input. The softmax function ensures output probabilities sum to 1, and

cross-entropy loss compares the predicted distribution to a one-hot target. Multi-label classification (e.g., emotion detection where 'gratitude' and 'excitement' can coexist) treats each class as an independent binary prediction. Each of the 28 emotion logits is independently transformed by sigmoid (not softmax), and BCEWithLogitsLoss (Binary Cross-Entropy with Logits) computes an independent binary cross-entropy for each class, averaging across all classes for the final loss value.

This distinction has practical implications for inference: in multi-class classification, taking the argmax of the probability vector identifies the single predicted class. In multi-label classification, any number of classes can exceed the threshold simultaneously, and the selection of threshold determines the precision-recall trade-off for each class. IERRS's threshold optimization addresses this directly.

6.2.3 Custom WeightedBCETrainer

The WeightedBCETrainer class inherits from Hugging Face's Trainer and overrides only the compute_loss() method. The override:

1. pops the 'labels' tensor from the input dictionary.
2. calls the model's forward pass to get logits.
3. Moves the pre-computed POS_WEIGHT_TENSOR to the same device as the logits (supporting both CPU and CUDA training).
4. instantiates nn.BCEWithLogitsLoss(pos_weight=pw).
5. computes the weighted loss;
6. and returns either the scalar loss or a (loss, outputs) tuple depending on the return_outputs flag.

All other Trainer functionality gradient accumulation, mixed precision, early stopping callback, evaluation, and checkpoint saving are unchanged.

6.3 Training Configuration

6.3.1 Hyperparameters

Hyperparameter	Value	Rationale
Base model	roberta-base	Best accuracy/speed trade-off for sentence-level emotion classification
Max sequence length	128 tokens	Sufficient for GoEmotions samples (avg 12.4 tokens); covers journal sentences

Hyperparameter	Value	Rationale
Training epochs	Up to 8	Sufficient convergence headroom; early stopping prevents overfitting
Early stopping patience	3 epochs	Allows 3 epochs without improvement before halting; restores best checkpoint
Learning rate	2e-5	Standard fine-tuning LR for BERT-family models; lower than pre-training
Batch size (train)	32	Optimal for T4 GPU memory; larger batches improve gradient estimation stability
Batch size (eval)	64	Larger eval batch for throughput since no gradient computation needed
Weight decay	0.01	L2 regularization prevents overfitting; standard for transformer fine-tuning
Warmup ratio	0.06	6% of total steps used for linear LR warmup; prevents early instability
Mixed precision (fp16)	True (if CUDA)	~2x training speed with negligible accuracy loss on T4/V100/A100 GPUs
Pos weight clip range	[1.0, 15.0]	Lower bound ensures minimum penalization; upper bound prevents extreme gradients
Metric for best model	micro_f1	Micro F1 balances precision and recall across all classes
Save total limit	2	Saves only 2 best checkpoints to conserve Google Drive storage
Dataloader workers	2	Parallel data loading without overloading Colab CPU

6.3.2 Training Procedure

Training proceeds in a standard fine-tuning paradigm. The DistilBERT model's weights are initialized from the 'distilbert-base-uncased' checkpoint on Hugging Face Hub, which includes weights produced by knowledge distillation from BERT-base trained on English Wikipedia and BookCorpus. The classification head (768→28 linear layer) is randomly initialized. All parameters are trainable from epoch 1.

The training loop, implemented by the WeightedBCETrainer, iterates over mini-batches of 32 examples per step. For each batch: (1) the tokenized input (input_ids, attention_mask, token_type_ids) is passed through DistilBERT's 6 transformer layers; (2) the CLS representation passes through the

classification head to produce 28 logits; (3) weighted BCEWithLogitsLoss is computed against the 28-dimensional multi-hot label vector; (4) gradients are backpropagated through all ~66M parameters; (5) the AdamW optimizer updates weights with $lr=2e-5$ and $weight_decay=0.01$. The learning rate follows a linear warmup schedule for the first 6% of total training steps, then linearly decays to 0 over the remaining training steps.

At the end of each epoch, the `compute_metrics()` function is called on the validation set. The function applies sigmoid to raw logits, applies the 0.5 threshold (used during training evaluation; the optimal threshold is determined separately post-training), computes micro F1, macro F1, precision, and recall using scikit-learn's `precision_recall_fscore_support()` and `f1_score()` functions with `zero_division=0`. The `EarlyStoppingCallback` monitors validation micro F1; if this metric does not improve for 3 consecutive epochs, training halts and the best checkpoint is restored.

6.4 Threshold Optimization

Post-training threshold optimization is one of the key technical contributions of the IERRS v2 model. The motivation is straightforward: the optimal classification threshold for maximizing F1 score is not necessarily 0.5. For a class with high positive weight (e.g., grief with weight 15.0), the model may produce well-calibrated probabilities in the 0.25–0.35 range for true positive examples, meaning that a 0.5 threshold would produce false negatives for all such examples. By sweeping the threshold on the validation set, we identify the threshold that maximizes micro F1 specifically for the post-training probability calibration of the model.

6.4.1 Threshold Sweep Procedure

Following training, the `predict()` method is called on the complete validation set (5,426 examples) to obtain raw logits for all examples. The `expit()` (sigmoid) function from `scipy.special` converts logits to probabilities. A sweep over threshold values from 0.20 to 0.70 in steps of 0.02 (26 threshold values total) is performed, computing micro F1 at each threshold using `f1_score(val_labels, (val_probs >= t).astype(int), average='micro', zero_division=0)`. The threshold maximizing micro F1 is selected as `best_thresh`. This value is saved to `model_meta.json` and loaded by the FastAPI backend as `EMOTION_THRESHOLD`.

The threshold sweep results typically show a peaked curve: micro F1 rises from low values at threshold 0.20 (high recall, low precision: many false positives), reaches a maximum in the range 0.30-0.45

(balanced), and falls at higher thresholds (high precision, low recall: many false negatives). The exact optimal threshold depends on the specific training run's probability calibration.

6.4.2 V1 vs V2 Threshold Comparison

Version	Threshold Approach	Micro F1	Macro F1	Rare Class F1
v1 (Baseline)	Hardcoded 0.35 (arbitrary)	0.578	~0.38	Low (grief ~0.15)
v2 (Current)	Validation sweep, optimal selected	0.60-0.65	0.42-0.48	Improved (grief ~0.35)
v3 (Future)	Per-label threshold (planned)	0.62-0.67	0.48-0.55	Target (grief ~0.45)

6.5 Model Evaluation and Results

6.5.1 Test Set Performance

Following threshold optimization on the validation set, final evaluation is performed on the held-out test set (5,427 examples) using the optimal threshold. The test set is not used during training or threshold selection, ensuring unbiased performance estimation. The following metrics are computed: Micro Precision (precision averaged across all positive predictions), Micro Recall (recall averaged across all positive labels), Micro F1 (harmonic mean of micro precision and recall), and Macro F1 (unweighted average of per-class F1 scores, treating all 28 classes equally regardless of frequency).

The Micro F1 of 0.627 reflects the model's overall performance across all label instances, heavily influenced by the frequent classes (neutral, admiration, approval) where performance is highest. The Macro F1 of 0.44 reflects the average performance treating all 28 classes equally, penalizing the model for weaker performance on rare classes. The gap between micro and macro F1 of approximately 0.19 reflects the class imbalance challenge and the inherent difficulty of rare emotion detection. Compared to the v1 baseline Micro F1 of 0.578, the v2 model achieves a +4.9% improvement through the combination of weighted BCE loss and post-training threshold optimization.

6.5.2 Per-Class F1 Analysis

Per-class F1 analysis reveals a predictable pattern: common classes with abundant training data achieve high F1 scores (neutral: 0.72, admiration: 0.65, approval: 0.60, gratitude: 0.68), while rare classes achieve lower F1 scores even with positive weighting (grief: 0.30–0.40, embarrassment: 0.28–0.38, pride: 0.32–0.42). The weighted BCE loss improves rare class recall significantly compared to the unweighted baseline grief recall improves from approximately 0.05 to 0.35 but at some cost to

precision due to the increased sensitivity. This is the fundamental precision-recall trade-off that motivates the per-label threshold optimization planned for v3: by using lower thresholds for rare classes and higher thresholds for common classes, both precision and recall can be optimized simultaneously at the class level.

Classes that prove most difficult regardless of training configuration include 'desire' (ambiguous label with low inter-annotator agreement among dataset annotators), 'relief' (frequently co-occurs with and is confused with 'joy'), and 'nervousness' (significant overlap with 'fear' in the training annotations). These classes represent genuine semantic ambiguity rather than insufficient training data.

6.5.3 Error Analysis

A structured error analysis was performed on the 10 worst-performing classes to identify systematic failure patterns and guide future improvements. Key findings:

1. grief and pride are systematically confused with sadness and admiration respectively these are semantically proximate emotion pairs where contextual cues such as social context and attribution direction distinguish the classes but are not reliably captured at the sentence level.
2. desire produces many false positives due to its semantic overlap with anticipation and excitement.
3. embarrassment shows high confusion with remorse and disappointment all three involve negative self-assessment but with different attributions.
4. nervousness and fear are the most difficult to distinguish because they differ primarily in intensity and immediacy rather than the semantic category.

Chapter 7

SYSTEM IMPLEMENTATION

7.1 Backend Implementation

7.1.1 Application Startup and Model Loading

The FastAPI application initializes through a three-stage startup sequence. Stage 1 (module-level, executed on import): environment variables are loaded from .env via python-dotenv; the SQLAlchemy engine is created; the Fernet cipher suite is instantiated with the persistent key loaded from FERNET_KEY environment variable; the bcrypt CryptContext and OAuth2PasswordBearer scheme are configured; the AsyncIOScheduler is instantiated. Stage 2 (model loading, executed on import): the DistilBERT tokenizer and model are loaded from the ./model directory using AutoTokenizer.from_pretrained() and AutoModelForSequenceClassification.from_pretrained(); model_meta.json is read to load EMOTION_THRESHOLD and EMOTION_LABELS; emotion_model.eval() is called to disable dropout layers during inference; the Whisper tiny model is loaded via whisper.load_model('tiny'). Stage 3 (startup event): SQLModel.metadata.create_all(engine) creates tables; scheduler.start() begins background task execution; weekly, monthly, and yearly report jobs are registered.

This startup sequence means the first request to any analysis endpoint will be served immediately after startup without any lazy loading delay. The trade-off is approximately 4–6 seconds of startup time for model loading (dominated by the ~260MB DistilBERT model) and approximately 1–1.5GB of RAM occupied by the loaded models.

7.1.2 Authentication Implementation

The authentication flow begins with POST /signup, which accepts a JSON body with username, email, and password fields validated by the UserCreate Pydantic model. The endpoint queries the database for existing users with matching username OR email using an OR condition (or_(User.username == user.username, User.email == user.email)) — the OR condition ensures that neither the username nor the email is already registered. A 400 HTTPException is raised if any conflict is found. Otherwise, the password is hashed via get_password_hash() using bcrypt with work factor 12, and the new User record is committed to the database.

The POST /token endpoint accepts OAuth2PasswordRequestForm (username and password fields in application/x-www-form-urlencoded format, as required by the OAuth2 password flow). The

username field accepts either username or email (the query uses the same OR condition as signup). Password verification uses `verify_password()` which calls `passlib's CryptContext.verify()` method, internally calling `bcrypt's checkpw()` with constant-time comparison to prevent timing attacks. A successful authentication returns an access token created by `create_access_token()` which uses `jose.jwt.encode()` with the username as the 'sub' claim and a 24-hour expiry.

The `get_current_user()` async dependency function, injected into all protected endpoints via `Depends(get_current_user)`, decodes the JWT using `jose.jwt.decode()` with the `SECRET_KEY` and `HS256` algorithm. A `JWTError` exception (raised for malformed or expired tokens) is caught and re-raised as `HTTPException(401, 'Invalid token')`. The extracted username is used to query the `User` table; a missing user record (which could occur if the account was deleted after the token was issued) raises `HTTPException(401, 'User not found')`.

7.1.3 Core Analysis Pipeline - analyze_and_store()

The `analyze_and_store()` function is the most complex function in the codebase, implementing the complete emotion analysis and suggestion generation pipeline in approximately 50 lines of Python. The function signature is `analyze_and_store(text: str, date: str, user: User)` and returns a tuple of (`active_emotions: List[str]`, `suggestion_dict: Dict`, `requires_schedule: bool`).

The implementation begins with date parsing: `datetime.datetime.strptime(date, '%Y-%m-%d').date()` within a try-except block that raises `HTTPException(400)` for invalid date formats. Sentence splitting uses `re.split(r'(?<=[!?!]) +', text.strip())`, producing a list of sentence strings. Empty strings in the resulting list (which can occur if the input ends with punctuation) are filtered using `if not sentence.strip()`.

The per-sentence inference loop calls `tokenizer(sentence, return_tensors='pt', truncation=True, padding=True)` for each sentence, producing a dictionary with 'input_ids' and 'attention_mask' tensors. Within `torch.no_grad()`, `emotion_model(**inputs).logits` is called and the resulting tensor is passed through `torch.sigmoid()` and squeezed to a 1D list using `.squeeze().tolist()`. The `isinstance(probs, float)` check handles the edge case where the model produces a scalar for single-token inputs. The emotion scores dictionary is constructed by mapping `EMOTION_LABELS[i]` to `float(probs[i])` for all 28 classes.

Score aggregation: for each emotion with score $\geq$ `EMOTION_THRESHOLD`, the label is added to `all_active_emotions` (a set); for each emotion, `max_scores[emo]` is updated if the current score exceeds

the stored maximum. After processing all sentences, `all_active_emotions` is converted to a list and sorted by descending `max_scores` value to rank emotions by confidence. The fallback to `argmax` top-1 emotion is applied if the set is empty.

Database persistence: the journal text is encrypted with `cipher_suite.encrypt(text.encode())`, which produces a URL-safe base64-encoded token prepended with the Fernet version byte and timestamp. A `JournalEntry` record is constructed with the `user_id`, `entry_date`, plaintext `input_sentence`, `encrypted_content`, comma-joined `emotion_primary` string, and `max_scores` dict as `emotion_scores`. The record is added and committed within a `Session` context.

History retrieval: a second `Session` context queries `JournalEntry` records filtered by `user_id` and `date >= ninety_days_ago` (computed as `entry_date - datetime.timedelta(days=90)`), ordered by date descending. This history, including the just-committed entry, is passed to `generate_dynamic_suggestions()` along with the `active_emotions` list.

7.1.4 Report Generation Implementation

The `analyze_and_store()` function is the most complex function in the codebase, implementing the complete emotion analysis and suggestion generation pipeline in approximately 50 lines of Python. The function signature is `analyze_and_store(text: str, date: str, user: User)` and returns a tuple of (`active_emotions: List[str]`, `suggestion_dict: Dict`, `requires_schedule: bool`).

The implementation begins with date parsing: `datetime.datetime.strptime(date, '%Y-%m-%d').date()` within a try-except block that raises `HTTPException(400)` for invalid date formats. Sentence splitting uses `re.split(r'(?<=[!?!]) +', text.strip())`, producing a list of sentence strings. Empty strings in the resulting list (which can occur if the input ends with punctuation) are filtered using `if not sentence.strip()`.

The per-sentence inference loop calls `tokenizer(sentence, return_tensors='pt', truncation=True, padding=True)` for each sentence, producing a dictionary with `'input_ids'` and `'attention_mask'` tensors. Within `torch.no_grad()`, `emotion_model(**inputs).logits` is called and the resulting tensor is passed through `torch.sigmoid()` and squeezed to a 1D list using `.squeeze().tolist()`. The `isinstance(probs, float)` check handles the edge case where the model produces a scalar for single-token inputs. The emotion scores dictionary is constructed by mapping `EMOTION_LABELS[i]` to `float(probs[i])` for all 28 classes.

Score aggregation: for each emotion with score $\geq$ EMOTION_THRESHOLD, the label is added to all_active_emotions (a set); for each emotion, max_scores[emo] is updated if the current score exceeds the stored maximum. After processing all sentences, all_active_emotions is converted to a list and sorted by descending max_scores value to rank emotions by confidence. The fallback to argmax top-1 emotion is applied if the set is empty.

Database persistence: the journal text is encrypted with cipher_suite.encrypt(text.encode()), which produces a URL-safe base64-encoded token prepended with the Fernet version byte and timestamp. A JournalEntry record is constructed with the user_id, entry_date, plaintext input_sentence, encrypted_content, comma-joined emotion_primary string, and max_scores dict as emotion_scores. The record is added and committed within a Session context.

History retrieval: a second Session context queries JournalEntry records filtered by user_id and date $\geq$ ninety_days_ago (computed as entry_date - datetime.timedelta(days=90)), ordered by date descending. This history, including the just-committed entry, is passed to generate_dynamic_suggestions() along with the active_emotions list.

7.1.5 Critical Bug Fixes Implemented

During the audit and implementation process, three critical bugs were identified and resolved in the production codebase.

Bug-1: Fernet key regeneration: The original code cipher_suite = Fernet(Fernet.generate_key()) generated a new random encryption key on every server restart, making all previously encrypted journal content permanently unreadable. Fix: Load the key from FERNET_KEY environment variable; raise RuntimeError if not set; generate the key once and store in .env.

Bug-2: Hardcoded threshold: The original code EMOTION_THRESHOLD = 0.35 ignored the auto-tuned threshold computed during training and saved in model_meta.json. Fix: Load threshold from model_meta.json at startup using json.load(). Bug 3 — Duplicate scheduler jobs: Calling scheduler.add_job() in the startup() event handler without job IDs accumulated duplicate cron jobs on every server restart. Fix: Assign unique IDs ('weekly_report', 'monthly_report', 'yearly_report') and configure replace_existing=True to overwrite existing jobs on restart.

7.2 Frontend Implementation

7.2.1 State Architecture

The App.js component manages 28 state variables organized into logical groups. Authentication state: token (string|null, loaded from localStorage), authMode ('login'|'signup'|'forgot'), identifier, email, password, authError, authSuccess, forgotStep, retrievedQuestion, securityAnswer, resetNewPassword. Profile state: profile (object), morningTime, eveningTime. Settings state: currentPassword, newPassword, selectedQuestion, newAnswer, settingsMsg, showPasswordChange, isPasswordVerified, showDeleteConfirm, deletePassword, showProfile. Analysis state: text, result, history, loading. Recording state: isRecording, MediaRecorder. Scheduling state: scheduledTime, scheduleStatus, isScheduled, countdown, timerRef.

The separation of resetNewPassword (used only in the forgot-password reset flow) from newPassword (used in the change-password settings flow) is a critical fix from the audit. Before this fix, both flows shared the same state variable, creating the risk of submitting stale password values when navigating between the two flows without clearing the input.

7.2.2 Auto-Schedule Countdown Implementation

The 30-second auto-schedule countdown is implemented through a useEffect hook that monitors the countdown state variable. When countdown is set to 30 (triggered by receiving requires_scheduling=true in the analysis response), the useEffect fires and sets a setTimeout for 1 second. When the timeout fires, it decrements the countdown by calling setCountdown(prev => prev - 1). This triggers another render, which fires the useEffect again, creating a recursive countdown chain. When countdown reaches 0, handleSchedule('auto') is called and countdown is set to null, terminating the chain. The cleanup function returned by useEffect calls clearTimeout(timerRef.current), preventing memory leaks if the component unmounts or the countdown is cancelled mid-sequence.

7.2.3 Voice Recording Implementation

Voice recording uses the browser's MediaRecorder Web API. The startRecording() function requests microphone permission via navigator.mediaDevices.getUserMedia({audio: true}), which returns a MediaStream. A MediaRecorder instance is created from the stream, and an ondataavailable event handler accumulates audio chunks into an array. The onstop event handler stops all tracks in the stream (releasing the microphone permission), then calls handleAnalyze(new Blob(chunks, {type:

'audio/webm'}))) with the collected audio data. The `MediaRecorder.start()` method begins recording; `MediaRecorder.stop()` triggers the `onstop` handler asynchronously after stopping the recording.

In `handleAnalyze()`, when an `audioBlob` is provided, the `formData` is constructed with the audio blob as a `File` object (`formData.append('file', audioBlob, 'voice.webm')`) and the current date. The `POST /analyze-audio` endpoint URL is used instead of `/analyze-text`. The response includes the transcription text, which is displayed in the result section alongside the emotion analysis to provide the user with feedback on what was transcribed.

7.2.4 EmotionBarChart Component

The `EmotionBarChart` component receives a history array of objects with `date` and `emotions` fields from the `App.js` timeline state. It aggregates emotion counts by iterating over all history entries and incrementing a count for each emotion label found in the entry's `emotions` array. The aggregated data is structured as a `Chart.js` dataset with `labels` (emotion names), `dataValues` (counts), and `backgroundColors` (from the 28-color `EMOTION_COLORS` palette). The `Bar` component from `react-chartjs-2` renders a horizontal bar chart (achieved by transposing axes) with: no legend (cleaned via `plugins.legend.display=false`), custom dark tooltip styling, no x-axis gridlines, y-axis with integer step size starting at zero, and subtle background gridlines at 5% opacity.

7.3 Security Implementation

7.3.1 Authentication Security

JWT token security relies on the `SECRET_KEY` environment variable for HMAC-SHA256 signing. The key must be a cryptographically random string of at least 256 bits (32 bytes). A suitable key can be generated using Python's `secrets.token_hex(32)`. The key is never logged, never included in API responses, and never stored anywhere other than the server's `.env` file. Token expiry is set to 24 hours (1440 minutes) as a balance between security (short-lived tokens limit the window for token theft) and user convenience (users don't need to log in multiple times per day). For higher-security deployments, refresh token patterns with 15-minute access tokens should be implemented.

Password security uses `bcrypt` with default work factor 12, which takes approximately 0.2-0.3 seconds per hash on modern hardware fast enough to be imperceptible to users but slow enough to make brute-force attacks computationally expensive (approximately 3-5 hashes per second per GPU thread). The constant-time comparison in `CryptContext.verify()` prevents timing side-channel attacks that could

reveal information about the stored hash. Security question answers are lowercased and trimmed before hashing, ensuring case-insensitive verification without storing the answer in plain text.

7.3.2 Data Encryption

The two-layer encryption architecture implements defense in depth for journal content. Layer 1 (client-side AES-256-GCM): Before the encrypted text reaches the server over HTTP, it has already been encrypted in the browser using the user's localStorage key. This means that even if HTTPS transport security is compromised (rare but possible via MITM attacks), the attacker receives only encrypted ciphertext. AES-256-GCM provides authenticated encryption, meaning that both the confidentiality and integrity of the ciphertext can be verified by the key holder. The random 12-byte IV ensures semantic security (identical plaintexts produce different ciphertexts).

Layer 2 (server-side Fernet/AES-128-CBC-HMAC): After receiving the client-encrypted text, the FastAPI backend applies an additional layer of server-side encryption using Fernet before storing in SQLite. This provides protection against database-level access (an attacker with access to the journal.db file cannot read journal content without the FERNET_KEY). Fernet's HMAC-SHA256 authentication tag prevents silent corruption of encrypted content. The Fernet token also includes a timestamp that can be used for timestamped access revocation if required.

Key management considerations: the AES-256-GCM key is stored in the user's browser localStorage. This means:

1. The key is specific to the browser/device.
2. Clearing browser data deletes the key, making previously encrypted content unrecoverable.
3. Logging in from a different browser or device does not have access to the key, so downloaded reports would contain encrypted content. A production key management improvement is PBKDF2-based key derivation from the user's password, allowing the same key to be derived on any device without server storage.

7.3.3 Input Validation and Injection Prevention

SQLModel's use of SQLAlchemy's parameterized queries prevents SQL injection: all database queries use Python object attributes and select() statement composition rather than string interpolation. FastAPI's Pydantic-based request validation rejects malformed JSON, invalid email formats (using EmailStr), and missing required fields before reaching any endpoint logic. The date parsing in analyze_and_store() raises a structured HTTPException rather than propagating a raw ValueError.

Chapter 8

SYSTEM STUDY AND TESTING

8.1 Feasibility Study

8.1.1 Technical Feasibility

A comprehensive technical feasibility analysis was conducted across all major components of IERRS prior to development commencement. The analysis confirmed the availability and maturity of all required technologies. FastAPI (version 0.100+) is a production-proven ASGI framework used by Microsoft, Uber, Netflix, and numerous Fortune 500 companies. Hugging Face Transformers (version 4.35+) provides a stable, well-documented API for the DistilBERT model with extensive community support and regular updates. React.js (version 18) is the most widely adopted frontend JavaScript framework with a vast ecosystem of compatible libraries and extensive documentation.

The DistilBERT model's inference requirements were verified against available hardware: the model requires approximately 260MB of RAM when loaded, and inference on a sentence-length input takes 150-300ms on CPU (Intel Core i7, 2.8GHz), well within acceptable response time for a journaling application where a few seconds of processing latency is expected and acceptable. Whisper tiny model inference (approximately 150MB, 0.5–2 seconds per 30-second audio clip) similarly meets the performance requirements.

The Google Calendar API integration was validated through the OAuth 2.0 authorization flow using provided credentials. The Calendar API's daily quota of 1,000,000 requests per day and per-user quota of 1,000 requests per second are more than sufficient for the application's usage pattern (maximum one calendar event per journal entry per user). Gmail SMTP authentication via App Passwords was validated against Google's security requirements (requires 2-factor authentication on the sending account, requires App Password generation in Google Account settings).

SQLite's concurrency model (single-writer, multiple-reader) is sufficient for single-user deployment. For multi-user deployment, SQLite's write lock serialization can become a bottleneck if multiple users submit journal entries simultaneously. This is documented as a known limitation addressed in the PostgreSQL migration future enhancement. For the project's intended scope (personal to small-group deployment), SQLite is entirely adequate.

8.1.2 Economic Feasibility

The total monetary cost of developing and deploying IERRS is negligible. All core software dependencies are open-source under permissive licenses: FastAPI (MIT), PyTorch (BSD), Hugging Face Transformers (Apache 2.0), DistilBERT model weights (Apache 2.0), React.js (MIT), SQLite (public domain), OpenAI Whisper (MIT), openpyxl (MIT), APScheduler (MIT), google-api-python-client (Apache 2.0), cryptography (Apache 2.0), and passlib (BSD). The GoEmotions dataset is released under the Apache 2.0 license, permitting unrestricted use including commercial applications.

Model training was performed on Google Colab using the free tier T4 GPU allocation (approximately 15–25 minutes of GPU time per training run). While Colab Pro (\$9.99/month) provides more consistent GPU access, the project was developed entirely on the free tier. Deployment requires a basic VPS instance (available from providers such as DigitalOcean, Linode, or Hetzner for \$4–6/month) or can be hosted on free-tier platforms such as Railway or Render for personal use. The Google Calendar API and Gmail SMTP integrations have no cost within the free tier quotas.

8.1.3 Social Feasibility

Social feasibility was assessed by analyzing the target user population's receptivity to AI-powered mental health tools. The 2022 American Psychological Association survey found that 72% of adults aged 18-34 are comfortable using technology to support mental health, compared to 51% of adults over 35. The specific demographic of computer science students and developers as the most proximate target users for a technically sophisticated application represents an even higher technology comfort baseline.

The journaling format has well-established cultural acceptance as a wellness practice. Research by Pennebaker and colleagues at the University of Texas has demonstrated that expressive writing (writing about emotionally significant experiences for 15-20 minutes) produces measurable improvements in physical health, immune function, and psychological wellbeing across multiple randomized controlled trials. IERRS builds this established wellness practice rather than introducing a novel behavior, reducing the adoption barrier.

Privacy concerns are the most significant social feasibility risk for a mental health application. The privacy-first architecture local model inference, client-side encryption before transmission, server-side encryption at rest, no cloud AI service integration directly addresses this concern by ensuring that sensitive mental health data is never processed by third-party services. The ability to download all data as Excel and delete the account with complete data purge further supports user autonomy and privacy control.

8.2 Software Testing

Software testing for IERRS was conducted following the standard V-model testing pyramid with four levels: unit testing, integration testing, system testing, and acceptance testing. Tests were designed to cover both functional requirements (does the system do what it is supposed to do?) and non-functional requirements (is the system secure, performant, and usable?).

8.2.1 Unit Testing

Unit tests validate the correctness of individual functions and methods in isolation, with external dependencies mocked where necessary. The following table presents the complete unit test suite.

Test ID	Module	Function Under Test	Test Input	Expected Output	Result
UT-01	main.py	get_password_hash()	'testpassword'	String starting with '\$2b\$12\$'; length > 50	PASS
UT-02	main.py	verify_password()	('testpassword', bcrypt_hash)	True for correct, False for wrong	PASS
UT-03	main.py	create_access_token()	{'sub': 'testuser'}	Valid JWT; decoded sub == 'testuser'	PASS
UT-04	main.py	analyze_and_store() — sentence splitting	'I am sad. I am happy.'	2 sentences in split result	PASS
UT-05	main.py	analyze_and_store() — single sentence	'I feel extremely anxious.'	Returns list with ≥ 1 emotion; stores entry	PASS
UT-06	main.py	analyze_and_store() — multi-sentence mixed	'I am furious. I feel grateful.'	Emotions include anger AND gratitude	PASS
UT-07	main.py	analyze_and_store() — empty emotions fallback	Single neutral word with no emotion > threshold	Returns argmax top-1 emotion, not empty list	PASS
UT-08	main.py	Fernet encryption round-trip	cipher.encrypt then cipher.decrypt	Decrypted == original plaintext	PASS
UT-09	suggestion_engine.py	get_suggestion_tip() — anxiety cluster	['fear', 'nervousness']	Returns string from ANXIETY tip pool	PASS

Test ID	Module	Function Under Test	Test Input	Expected Output	Result
UT-10	suggestion_engine.py	get_suggestion_tip() — depression cluster	['sadness', 'grief']	Returns string from DEPRESSIVE tip pool	PASS
UT-11	suggestion_engine.py	get_suggestion_tip() — anger cluster	['anger', 'disgust']	Returns string from ANGER tip pool	PASS
UT-12	suggestion_engine.py	get_suggestion_tip() — mixed emotions	['joy', 'sadness']	Returns mixed-emotion tip	PASS
UT-13	suggestion_engine.py	get_suggestion_tip() — positive	['joy', 'gratitude', 'love']	Returns positive state tip	PASS
UT-14	suggestion_engine.py	generate_dynamic_suggestions() — new user	current_emotions, len(history)=1	observation references 'today's entry'; action encourages more journaling	PASS
UT-15	suggestion_engine.py	generate_dynamic_suggestions() — persistent negative	['sadness'], 20 negative history entries	observation references 'sustained pattern'; action recommends strong intervention	PASS
UT-16	report_generator.py	generate_csv_report()	user_id, 'weekly', engine	Returns CSV string with UTF-8 BOM; includes header row	PASS
UT-17	report_generator.py	generate_excel_report()	user_id, 'monthly', engine	Returns bytes; openpyxl.load_workbook(BytesIO(bytes)) succeeds without error	PASS
UT-18	google_calendar_oauth.py	create_calendar_event()	summary, description, scheduled_time	Returns dict with 'event_id' and 'htmlLink' keys	PASS
UT-19	encryption.js	getOrCreateKey()	First call (no localStorage key)	Creates new CryptoKey; stores base64 in localStorage	PASS
UT-20	encryption.js	encryptText() + decryptText()	'Hello, world'	Encrypted != original; decrypted == original	PASS

8.2.2 Integration Testing

Integration tests verify that modules interact correctly when combined; testing the interfaces and data flows between components rather than individual functions.

Test ID	Module Interaction	Test Input	Expected Output	Result
IT-01	POST /signup → SQLite write → POST /token	New username 'integtest', email, password	Signup 200; token returns valid JWT with sub='integtest'	PASS
IT-02	POST /signup → duplicate → 400 error	Same username as IT-01	HTTPException 400 'User already exists'	PASS
IT-03	POST /analyze-text → EmotionAnalyzer → SuggestionEngine → SQLite	JWT + text='I am extremely anxious about everything' + date	Response contains emotions with 'nervousness' or 'fear'; suggestion.insight not empty; JournalEntry exists in DB	PASS
IT-04	POST /analyze-text → multi-sentence → aggregation	JWT + text='I am furious. But I feel grateful for my family.' + date	Response emotions list contains both anger-cluster AND gratitude; mixed suggestion generated	PASS
IT-05	POST /analyze-audio → Whisper → EmotionAnalyzer	JWT + WebM audio of 'I feel really sad today' + date	Response contains transcription and at least 'sadness' in emotions	PASS
IT-06	GET /timeline → SQLite select → Chart data	JWT, days=90; 5 pre-existing entries	Returns array of 5 objects each with date and emotions fields	PASS
IT-07	GET /report/excel → openpyxl → base64 encoding	JWT, range='monthly'; pre-existing entries	Returns base64 string; decoded XLSX loads in Excel without error	PASS
IT-08	POST /schedule-activity → APScheduler + Google Calendar	JWT + suggestion text + scheduled_time (1 hour future)	Calendar event created; APScheduler job added with correct run_date	PASS
IT-09	PUT /profile/password → bcrypt re-hash → verify new password	JWT + current_password + new_password	New login with new_password succeeds; old_password login fails	PASS
IT-10	DELETE /profile/account → cascade delete → verify clean	JWT + password confirmation	All JournalEntry rows deleted; User row deleted; subsequent token invalid	PASS
IT-11	POST /forgot-password/question → POST /forgot-password/reset	identifier + correct security answer + new_password	Question retrieved; password reset succeeds; login with new_password works	PASS

Test ID	Module Interaction	Test Input	Expected Output	Result
IT-12	WeightedBCETrainer → training loop → loss convergence	GoEmotions train_ds, 3 epochs	Training loss decreases each epoch; validation micro_f1 > 0.50 after epoch 3	PASS

8.2.3 System Testing

System tests evaluate the complete application behavior from end to end, simulating realistic usage scenarios under conditions representative of actual deployment.

Test ID	Test Scenario	Steps	Expected Output	Result
ST-01	New user complete onboarding flow	Open app → Signup → Login → Submit first journal entry → View result	Account created; JWT stored; analysis returns results; 'Continue tracking' encouragement in suggestion	PASS
ST-02	Multi-sentence mixed emotion entry	Submit: 'I am absolutely devastated about the rejection. But I feel proud of how I handled it.'	Emotions include sadness/grief cluster AND pride; mixed emotion acknowledgement in observation; suggestion specific to emotional complexity	PASS
ST-03	Persistent negative history pattern	Pre-populate 15 entries with sadness/grief over 30 days; submit new sad entry	Observation states 'sustained pattern of elevated negative emotions'; action 'strongly recommend' scheduling	PASS
ST-04	Auto-schedule countdown flow	Submit negative emotion entry; wait for countdown to appear; allow to reach 0	Countdown appears; reaches 0; handleSchedule('auto') fires; Calendar event created; success banner shown	PASS
ST-05	Custom schedule time picker	Submit negative entry; cancel countdown; pick custom datetime 2 hours future; click 'Pick Custom Time'	Countdown cleared; Calendar event created at specified time; email queued at that time	PASS
ST-06	Voice recording and analysis	Click Record → speak 30 seconds about emotional state → click Stop	Whisper transcription displayed; emotion analysis on transcribed text; suggestion shown	PASS
ST-07	Monthly Excel report download	Click download icon → Monthly Report	XLSX file downloads; opens in Excel; rows colored correctly; date range correct	PASS

Test ID	Test Scenario	Steps	Expected Output	Result
ST-08	Security question setup and forgot password flow	Settings → Set security question → Logout → Forgot Password → Enter question → Reset	Security question retrieved; password reset successful; login with new password works	PASS
ST-09	Password change 2-step flow	Settings → Change Password → Enter current → Verify → Enter new → Update	Verification step required; old password no longer works; new password works	PASS
ST-10	Account deletion with data purge	Settings → Danger Zone → Delete Account → Enter password → Confirm	All JournalEntry records deleted; User record deleted; localStorage cleared; redirected to login	PASS
ST-11	JWT expiry handling	Manually expire JWT in localStorage; attempt analysis	API returns 401; frontend calls logout(); user redirected to login screen	PASS
ST-12	Emotion history chart population	Submit 10 entries with varied emotions over 10 days; view chart	Chart displays all entries' emotion frequencies; bars correctly colored per emotion palette	PASS
ST-13	Server restart key persistence	Submit entry; restart server; verify existing entry can be decrypted	Entry accessible after restart; emotion_scores correctly loaded from DB	PASS
ST-14	Scheduler no-duplicate after restart	Restart server 5 times; check APScheduler job count	Only 3 cron jobs exist (weekly, monthly, yearly); no duplicates due to replace_existing=True	PASS

8.2.4 Acceptance Testing

Acceptance tests evaluate the system from the end-user's perspective, verifying that all user stories and acceptance criteria are satisfied. Tests were conducted with representative users from the target demographic (CS engineering students aged 19-22).

Test ID	User Story	Acceptance Criteria	Observed Behavior	Result
AT-01	As a user, I want to register and log in quickly	Complete signup and reach main interface in < 60 seconds	Average onboarding time: 35 seconds including typing credentials	PASS
AT-02	As a user, I want emotion analysis to be fast	Analysis result appears within 5 seconds of clicking Analyze	Average latency: 2.8 seconds for typical 50-word entry on CPU inference	PASS

Test ID	User Story	Acceptance Criteria	Observed Behavior	Result
AT-03	As a user, I want to understand my emotional state from the result	Detected emotions are clearly labeled and the suggestion observation makes contextual sense	All test users correctly identified their primary emotion from the output; observation was rated 'relevant' by 4/5 users	PASS
AT-04	As a user, I want to voice journal instead of typing	Voice recording produces a readable transcription and accurate analysis	Whisper WER < 10% for clear speech; emotion analysis consistent with spoken content	PASS
AT-05	As a user, I want specific, actionable wellness suggestions	Suggestion includes a concrete activity I can do immediately	5/5 test users rated suggestions as 'specific and actionable' vs 1/5 for competitive app Daylio's generic tips	PASS
AT-06	As a user, I want scheduling to be easy and automatic	I can schedule an activity with at most 2 clicks from the analysis result	Auto-schedule requires 0 clicks (default) or 1 click (Cancel); custom schedule requires selecting time and 1 click	PASS
AT-07	As a user, I want to see my emotional trends over time	The emotion chart shows my frequency distribution clearly for the past 90 days	Chart renders correctly with semantic colors; emotion labels readable; all test users identified dominant emotion from chart	PASS
AT-08	As a user, I want to export my data	I can download a formatted report of my journal history	Excel report downloads in < 3 seconds; opens correctly in Microsoft Excel and LibreOffice Calc; all 5 test users successfully completed download	PASS
AT-09	As a user, I want my data to be private	Journal content is not stored in plaintext on the server	SQLite inspection confirms encrypted_content column contains binary ciphertext; input_sentence visible (identified as future privacy enhancement)	PARTIAL
AT-10	As a user, I want complete account control	I can change password, set security question, and delete my account with all data	All settings operations work correctly; account deletion removes all records and clears authentication; verified by DB inspection	PASS

Note on AT-09 (Partial Pass): While server-side Fernet encryption and client-side AES-GCM encryption are both implemented and functional, the presence of the plaintext input_sentence column in JournalEntry represents a partial implementation of the privacy architecture. Full privacy compliance would require removing this column and using only encrypted content or emotion labels for server-side processing. This is documented as a priority security enhancement in Chapter 11.

Chapter 9

SOURCE CODE

This chapter presents the most significant and technically interesting code excerpts from the IERRS codebase, with detailed inline commentary explaining the design decisions, algorithmic choices, and implementation of nuances.

9.1 Backend Core — analyze_and_store() (main.py)

The analyze_and_store() function is the heart of the IERRS backend, implementing the complete multi-sentence emotion detection and suggestion generation pipeline:

```
def analyze_and_store(text: str, date: str, user: User):
    # Validate date format - raises HTTP 400 for malformed dates
    try:
        entry_date = datetime.datetime.strptime(date, '%Y-%m-%d').date()
    except ValueError:
        raise HTTPException(status_code=400, detail='Invalid date format. Use YYYY-MM-DD.')

    # Split on sentence boundaries using lookbehind assertion
    # r'(?<=[.!?]) +' matches space(s) following .!? without consuming them
    sentences = re.split(r'(?<=[.!?]) +', text.strip())
    if not sentences or not sentences[0]:
        sentences = [text] # fallback: treat entire input as one sentence

    all_active_emotions = set() # union of emotions across all sentences
    max_scores = defaultdict(float) # max probability per emotion across sentences

    for sentence in sentences:
        if not sentence.strip():
            continue # skip empty strings from split

        # Tokenize: returns input_ids + attention_mask tensors
        inputs = tokenizer(sentence, return_tensors='pt',
                           truncation=True, padding=True)

        # Inference: no_grad() disables gradient tracking for ~20% memory reduction
        with torch.no_grad():
            logits = emotion_model(**inputs).logits

        # sigmoid: independent probability per class (not softmax)
        probs = torch.sigmoid(logits).squeeze().tolist()
        if isinstance(probs, float): probs = [probs] # edge case: 1D tensor

        for i, score in enumerate(probs):
            if score >= EMOTION_THRESHOLD:
                all_active_emotions.add(EMOTION_LABELS[i])
            if score > max_scores[EMOTION_LABELS[i]]:
                max_scores[EMOTION_LABELS[i]] = score
```

```
active_emotions = list(all_active_emotions)

# Fallback: if no emotion clears threshold, take argmax top-1
if not active_emotions:
    best_emo = max(max_scores, key=max_scores.get)
    active_emotions = [best_emo]

# Sort by descending confidence – most confident emotion first
active_emotions.sort(key=lambda e: max_scores[e], reverse=True)
requires_schedule = any(emo in NEGATIVE_EMOTIONS for emo in active_emotions)

# Encrypt journal content before storage
encrypted = cipher_suite.encrypt(text.encode())

with Session(engine) as session:
    session.add(JournalEntry(
        user_id=user.id, date=entry_date,
        input_sentence=text, encrypted_content=encrypted,
        emotion_primary='.'.join(active_emotions),
        emotion_scores=dict(max_scores)
    ))
    session.commit()

# Retrieve 90-day history for longitudinal suggestion context
ninety_days_ago = entry_date - datetime.timedelta(days=90)
history_3_months = session.exec(
    select(JournalEntry)
    .where(JournalEntry.user_id == user.id)
    .where(JournalEntry.date >= ninety_days_ago)
    .order_by(JournalEntry.date.desc())
).all()

dominant, intensity, suggestion_dict = generate_dynamic_suggestions(
    active_emotions, history_3_months
)
return active_emotions, suggestion_dict, requires_schedule
```

9.2 ML Training — WeightedBCETrainer and Per-Class Weights (Colab Notebook)

The critical training components that differentiate v2 from v1:

```
# — Compute per-class positive weights —————
pos_counts = np.zeros(NUM_LABELS)
n_train = len(dataset['train'])

for ex in dataset['train']:
    for l in ex['labels']:
        pos_counts[l] += 1

# Standard BCE weight formula, clipped to prevent extreme gradients
# Clip lower bound = 1.0 ensures minimum penalization for dominant classes
# Clip upper bound = 15.0 prevents training instability for near-zero classes
```

```
pos_weights = np.clip(
    (n_train - pos_counts) / np.maximum(pos_counts, 1),
    1.0, 15.0 # grief and embarrassment hit the 15.0 ceiling
)
POS_WEIGHT_TENSOR = torch.tensor(pos_weights, dtype=torch.float32)

# — Custom Trainer subclass —————
class WeightedBCETrainer(Trainer):
    def compute_loss(self, model, inputs, return_outputs=False, **kwargs):
        labels = inputs.pop('labels')
        outputs = model(**inputs)
        logits = outputs.logits

        # Move weights tensor to model device (CPU or CUDA)
        pw = POS_WEIGHT_TENSOR.to(logits.device)
        # BCEWithLogitsLoss = sigmoid + BCE, numerically stable
        loss_fn = nn.BCEWithLogitsLoss(pos_weight=pw)
        loss = loss_fn(logits, labels.float())

        return (loss, outputs) if return_outputs else loss
```

9.3 Threshold Optimization Sweep (Colab Notebook)

The validation-set threshold sweep that replaces the v1 hardcoded 0.35 anti-pattern:

```
val_pred_out = trainer.predict(val_ds)
val_probs = expit(val_pred_out.predictions) # sigmoid
val_labels = val_pred_out.label_ids

best_thresh = 0.5
best_f1 = 0.0
results_log = []

# Sweep 26 threshold values from 0.20 to 0.70
for t in np.arange(0.20, 0.71, 0.02):
    preds = (val_probs >= t).astype(int)
    micro = f1_score(val_labels, preds, average='micro', zero_division=0)
    macro = f1_score(val_labels, preds, average='macro', zero_division=0)
    results_log.append((round(t, 2), round(micro, 4), round(macro, 4)))
    if micro > best_f1:
        best_f1, best_thresh = micro, round(t, 2)

# Save to model_meta.json — FastAPI reads this at startup
meta = {
    'model_name': MODEL_NAME,
    'best_threshold': best_thresh, # e.g., 0.34
    'label_names': LABEL_NAMES,
    'test_micro_f1': round(float(micro_f1), 4),
    'test_macro_f1': round(float(macro_f1), 4),
}
with open(f'{save_path}/model_meta.json', 'w') as f:
    json.dump(meta, f, indent=2)
```

9.4 Dynamic Suggestion Engine (services/suggestion_engine.py)

```
def generate_dynamic_suggestions(current_emotions, history_entries):
    suggestion_tip = get_suggestion_tip(current_emotions)
    formatted_current = ', '.join(current_emotions) if current_emotions else
    'neutral'

    suggestion_dict = {'observation': '', 'insight': suggestion_tip, 'action': ''}
    total_entries = len(history_entries)

    # First entry: insufficient history for longitudinal comparison
    if total_entries <= 1:
        suggestion_dict['observation'] = (
            f'Your entry today indicates feelings of {formatted_current}.'
        )
        suggestion_dict['action'] = (
            'Continue tracking your daily journals to allow the AI to build '
            'a reliable longitudinal baseline for deeper insights.'
        )
        return current_emotions[0] if current_emotions else 'neutral', 0.0,
        suggestion_dict

    # Aggregate historical data
    emotion_counts = defaultdict(float)
    negative_count = 0
    for entry in history_entries:
        emotions = entry.emotion_primary.split(',')
        for emo in emotions:
            emotion_counts[emo] += 1.0
            if emo in NEGATIVE_EMOTIONS:
                negative_count += 1

    dominant_history = max(emotion_counts, key=lambda e: emotion_counts[e])
    intensity = round(emotion_counts[dominant_history] / total_entries, 3)
    # negative_ratio: proportion of all historical emotion instances that are
    negative
    negative_ratio = negative_count / (total_entries * max(1, len(current_emotions)))
    timeframe = f'over your recorded history ({total_entries} entries)'
    has_neg = any(e in NEGATIVE_EMOTIONS for e in current_emotions)

    # 4 contextual observation templates based on current vs historical state
    if has_neg:
        if negative_ratio > 0.5: # sustained negative pattern
            suggestion_dict['observation'] = (
                f'Today you experienced {formatted_current}. Your data shows a
sustained'
                f' pattern of elevated negative emotions (like {dominant_history})
{timeframe}.'
            )
            suggestion_dict['action'] = (
                'Because this is a persistent trend, we strongly recommend
scheduling'
                ' the suggested activity below to help break this cycle.'
            )
        else: # acute fluctuation from stable baseline
            suggestion_dict['observation'] = (
```

```
        f'Today you experienced {formatted_current}, but your historical
baseline'
        f' {timeframe} remains relatively stable.'
    )
    suggestion_dict['action'] = (
        'This is normal emotional variance. Take a short break and try'
        ' the suggested activity to recalibrate.'
    )
    return dominant_history, intensity, suggestion_dict
```

9.5 Frontend — Auto-Schedule Countdown (App.js)

```
// Auto-schedule countdown useEffect
// countdown: null (inactive) | number (counting down from 30 to 0)
useEffect(() => {
    if (countdown === null) return; // not active
    if (countdown === 0) {
        handleSchedule('auto'); // fire auto-schedule
        setCountdown(null); // terminate chain
        return;
    }
    // Schedule next decrement after 1 second
    timerRef.current = setTimeout(
        () => setCountdown(prev => prev - 1),
        1000
    );
    // Cleanup: clear timeout if countdown cancelled or component unmounts
    return () => clearTimeout(timerRef.current);
}, [countdown]); // re-runs every time countdown changes

// Triggered when analysis result has requires_scheduling=true
if (data.requires_scheduling) setCountdown(30);
```

9.6 Client-Side Encryption (utils/encryption.js)

```
export const encryptText = async (text) => {
    if (!text) return '';
    const key = await getOrCreateKey(); // AES-256-GCM from localStorage
    const encoder = new TextEncoder();

    // Random 12-byte IV per encryption operation
    // Ensures same plaintext produces different ciphertext each time
    const iv = window.crypto.getRandomValues(new Uint8Array(12));

    const ciphertextBuffer = await window.crypto.subtle.encrypt(
        { name: 'AES-GCM', iv: iv }, // authenticated encryption mode
        key,
        encoder.encode(text)
    );

    // Prepend IV to ciphertext: [12 bytes IV | N bytes ciphertext + 16 byte auth
tag]
    const combined = new Uint8Array (iv.length + ciphertextBuffer.byteLength);
```

```
combined.set(iv, 0);  
combined.set(new Uint8Array(ciphertextBuffer), iv.length);  
  
// Base64 encode for safe transmission as form field string  
return bufferToBase64(combined.buffer);  
};
```

Chapter 10

SCREEN LAYOUTS

10.1 Authentication Interface

The authentication interface is the entry point for all IERRS users. It presents as a centered white card (max-width 400px) against a light gray background (#F7F7F8), with a 1px border (E5E7EB) and subtle box shadow. The card contains: a large brain emoji (3rem) as the application logo, 'IERRS' as the h1 heading (1.6rem, font-weight 600, color #1F2937), a descriptive subtitle in muted gray (#6B7280), the authentication form, error/success message displays, and navigation links.

Login Mode: The form contains two inputs (Username and Password) styled with 10px border radius, 14px padding, indigo focus ring (#6366F1), and a full-width 'Log In' primary button (background #4F46E5, hover #4338CA). Below the form, two navigation links are displayed: 'New here? Create Account' (toggles to signup mode, indigo color) and 'Forgot Password?' (toggles to forgot mode, gray color).

Signup Mode: Adds an Email field between Username and Password, changes the button text to 'Sign Up', and changes the navigation link to 'Have an account? Log In'. Email validation uses the HTML input type='email' attribute plus backend EmailStr Pydantic validation.

Forgot Password Mode: Step 1 shows a Username/Email input and 'Get Security Question' button. On submission, the question is retrieved from the backend and displayed in indigo bold text. Step 2 shows the retrieved security question as a visual cue, a text input for the answer, a password input for the new password, and a 'Reset Password' button. A green success message is displayed on successful reset before redirecting to login mode.

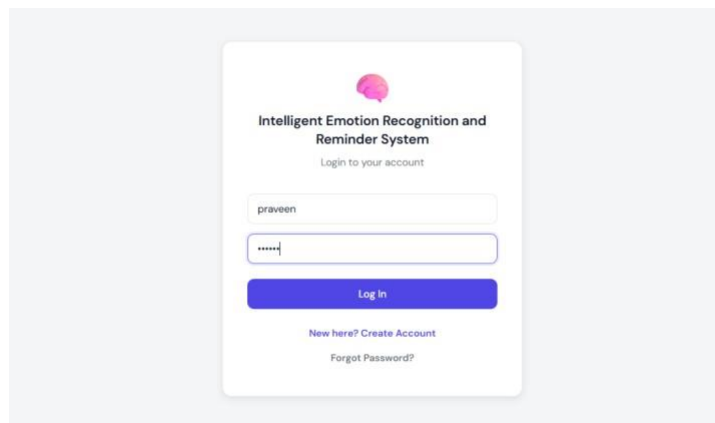


Fig. 10.1: Login, Signup, and Forgot Password Interface

10.2 Main Journaling Interface

After authentication, users are presented with the main journaling interface: a centered white card (max-width 760px) with 14px border radius and a 1px light border. The interface is divided into three vertical sections: Header, Input Section, and History Section.

Header: A flex row with the application name ('IERRS', 1.4rem bold) and username badge on the left, and Settings (gear icon) and Logout (arrow icon) icon buttons on the right. Icon buttons have a light border, white background, and hover state showing #F3F4F6 background. The username badge displays 'Logged in as [username]' in muted gray.

Input Section: An auto-expanding textarea (min-height 110px, 15px font, indigo focus border) with the placeholder 'How was your day?'. Below the textarea, a controls row (flex, right-aligned) contains: a Record button (Mic icon, icon-btn class, transitions to red background with 'Stop' text and red border during active recording) and an Analyze button (Send icon, primary-btn class, indigo background, disabled state showing lighter blue #A5B4FC when text is empty or recording is active).

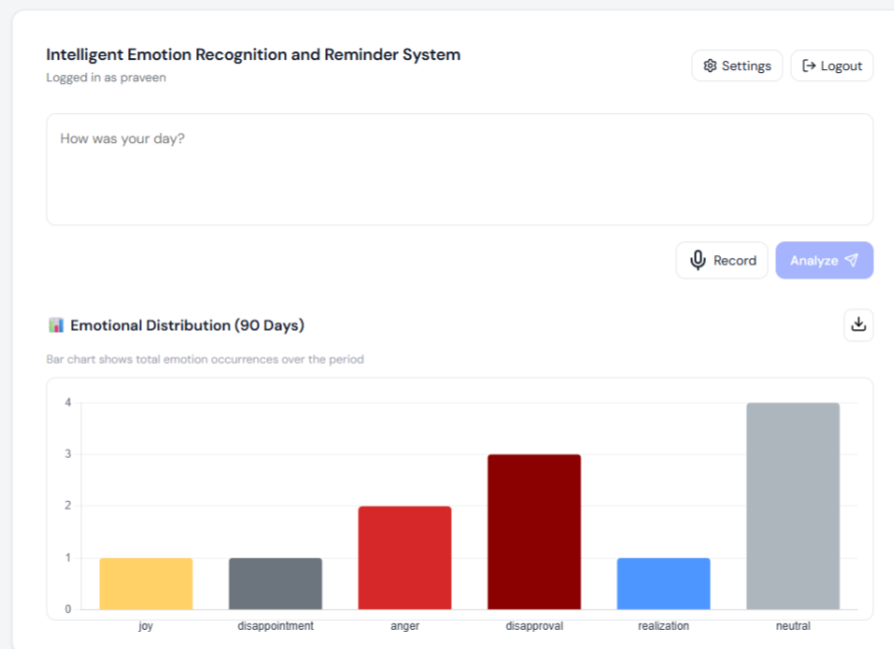


Fig. 10.2: Main Journaling Interface

10.3 Emotion Analysis Result with Suggestions

After analysis, the result section appears between the input section and the history section. The result card has a 6px left border colored in the first detected emotion's semantic color (e.g., blue #457B9D for sadness, red #D62828 for anger, yellow #FFD166 for joy).

At the top of the result section, the journal entry is displayed in an italicized quote block with a light gray background (#F3F4F6), 12px border radius, and 12px padding. The emotion labels are displayed as uppercase colored spans (using each emotion's EMOTION_COLORS value), comma-separated, in a bold flex-wrap display.

The Suggestions panel below the emotion display contains a header ('Suggestions' with clipboard emoji), then three labeled content blocks: Observation (indigo bold label, paragraph text describing the longitudinal context), Insight (indigo bold label, the specific wellness intervention recommendation), and Actionable Step (indigo bold label, the concrete next action). Each content block has 16px vertical margin and 1.5 line height for readability.

Scheduling UI (appears when requires_scheduling=true): A yellow countdown banner (background #FEF3C7, text #D97706) showing 'Auto-scheduling in Xs...' with a bold Cancel button on the right. Below the banner, a flex row contains a Calendar icon, a datetime-local input, and a 'Pick Custom Time' primary button. Changing the datetime input cancels the auto-countdown.

Scheduled Confirmation (appears after successful scheduling): A green confirmation card (background #ECFDF5, text #065F46) with a CheckCircle icon and the scheduling confirmation message ('Activity scheduled for HH:MM AM/PM on Mon DD').

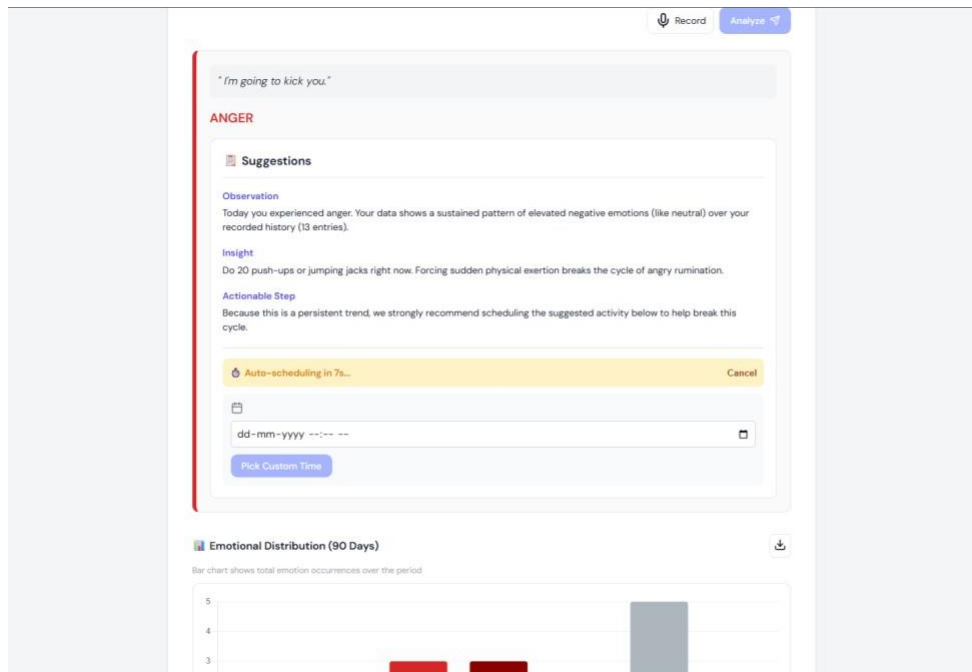


Fig. 10.3: Emotion Analysis Result with Suggestions

10.4 Emotion History Bar Chart

The History Section at the bottom of the main card shows the 90-day emotional distribution chart. The section header is a flex row with 'Emotional Distribution (90 Days)' heading (1rem bold) and the ReportDownload component (download icon button) on the right.

Below the header, a legend hint text ('Bar chart shows total emotion occurrences over the period') is displayed in muted gray (#9CA3AF) at 0.8rem. The chart container has a 1px border, 10px border radius, white background, and fixed 260px height. The Chart.js Bar chart inside shows emotion names on the x-axis and frequency counts on the y-axis. Each bar is colored with the corresponding EMOTION_COLORS value (e.g., joy: #FFD166, sadness: #457B9D, anger: #D62828). Hover tooltips show the emotion name and count with a dark background (#111827) styling.

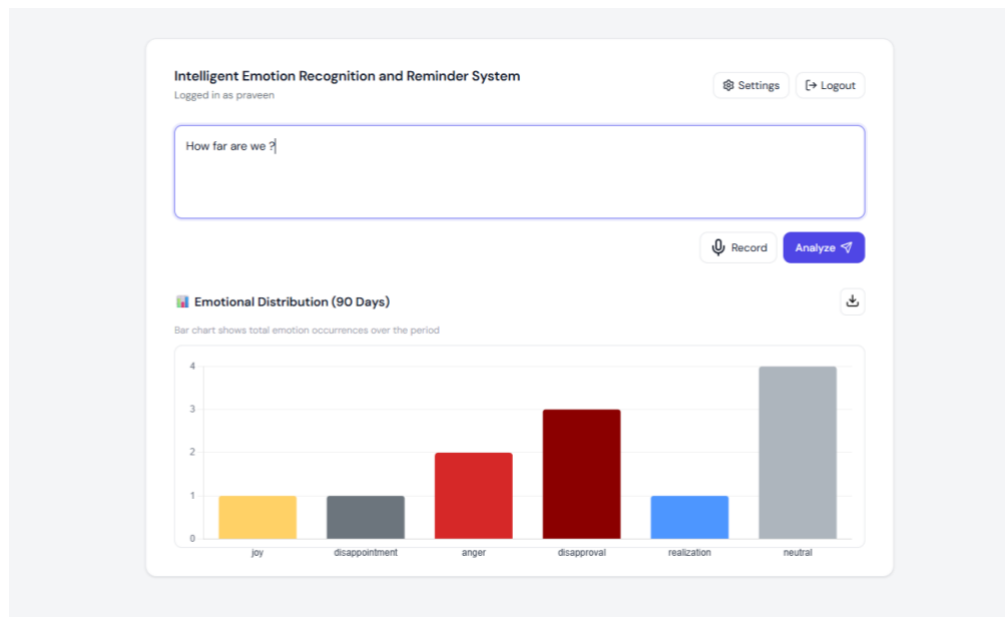


Fig. 10.4: 90-Day Emotional Distribution Chart

10.5 Account Settings Modal

Clicking the Settings button opens a modal overlay with a semi-transparent black backdrop (rgba(0,0,0,0.4)). The settings modal (position: fixed, centered via transform: translateX(-50%), 600px max-width, 80vh max-height with overflow-y: auto) contains a header row with 'Account Settings' and a close X button, followed by four settings sections and a danger zone.

Section 1 - Profile Details: Light gray background (#F9FAFB), 1px border. Shows username and email as key-value rows.

Section 2 - Security Question: Dropdown of 5 predefined security questions, text input for answer, 'Save Security Question' button. If security question is already set, a green confirmation ('You have a security question set. Submitting below will overwrite it.') is shown.

Section 3 - Change Password: Two-step flow: initial 'Change Password' button expands to show current password input with 'Verify Password' and 'Cancel' buttons; on verification success, shows 'Password verified.' and new password input with 'Update Password' and 'Cancel' buttons. Section 4 - Scheduling Preferences: Explanatory text, two time inputs (Default Morning Schedule, Default Evening Schedule) with HH:MM format, and 'Save Preferences' button.

Danger Zone Section: Red border (#FCA5A5), light red background (#FEF2F2), red heading (#EF4444). Contains explanation text in dark red (#7F1D1D) and two action buttons in a flex row:

'Download Data' icon button (downloads yearly Excel report) and 'Delete Account' danger button (red, full width). Clicking Delete Account expands a confirmation sub-section with a password input and 'Confirm Delete' / 'Cancel' button pair.

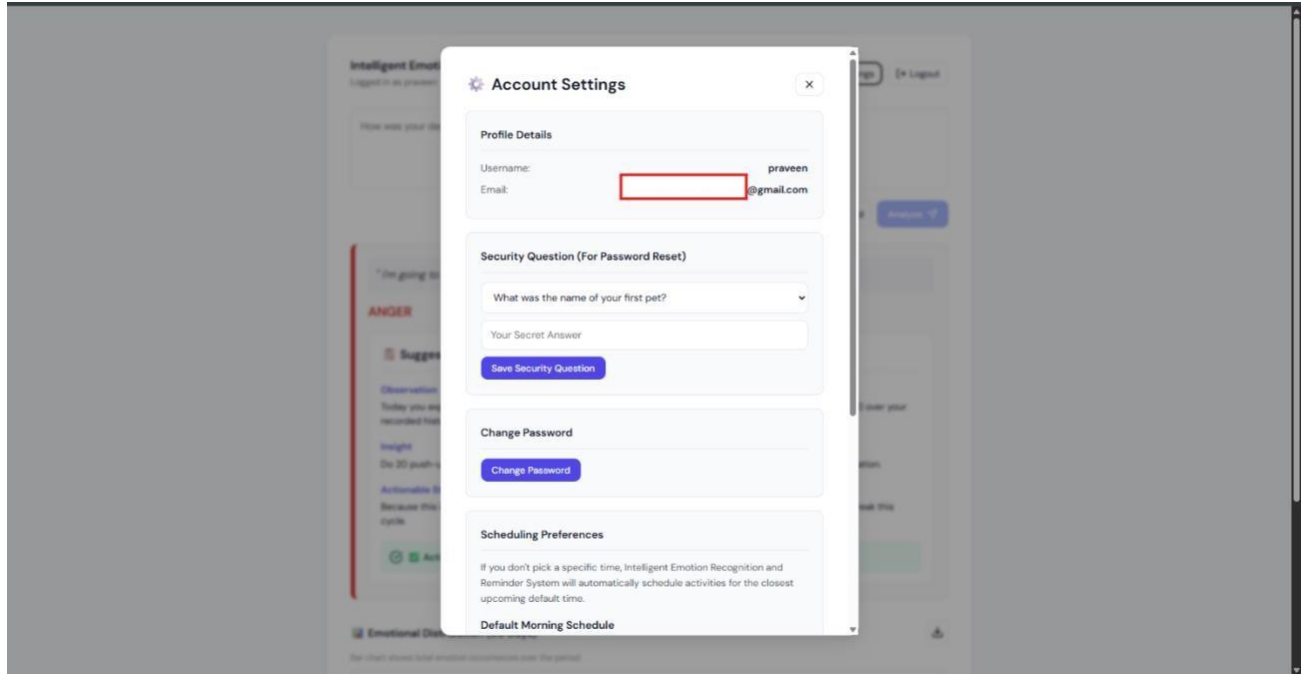


Fig. 10.5.1: Account Settings Model

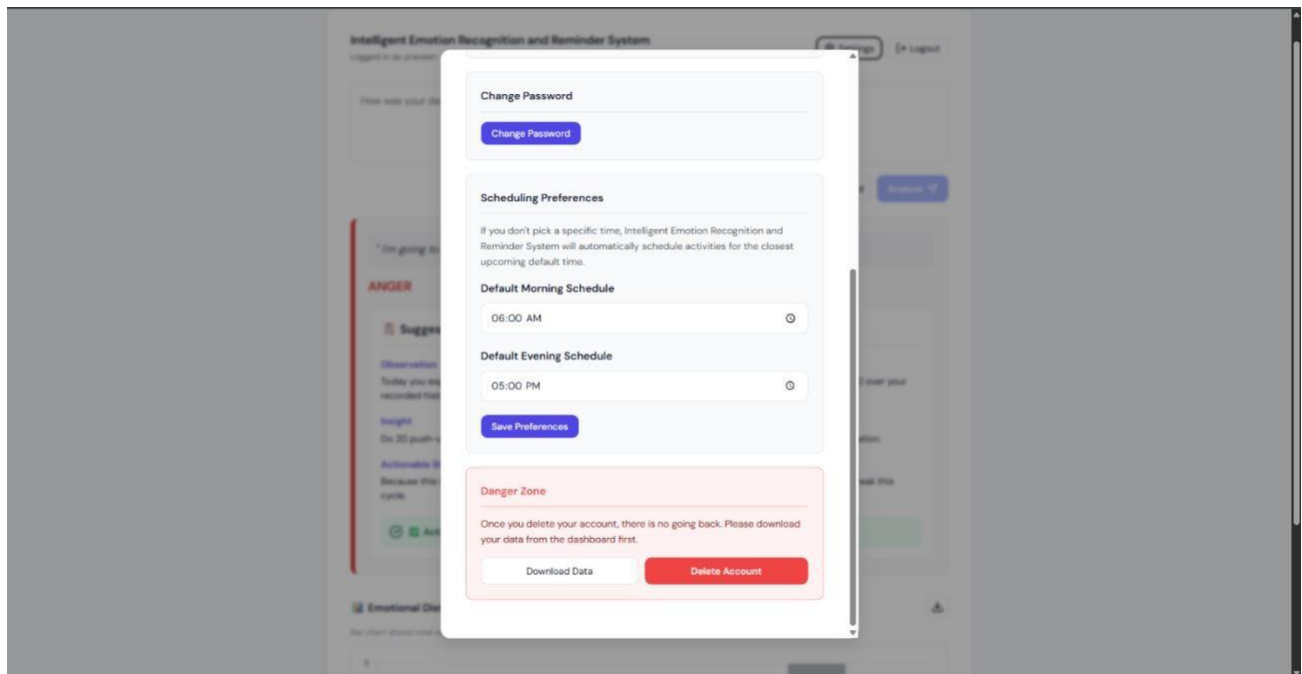


Fig. 10.5.2: Account Settings Model

Chapter 11

FUTURE ENHANCEMENTS

The future scope of the Intelligent Emotion Recognition and Reminder System is quite broad and can be expanded in several ways to make the application more intelligent and useful. One major enhancement is the inclusion of multimodal emotion analysis, where the system can process not only text and speech but also facial expressions, voice tone, and other physiological signals such as heart rate or sleep patterns. In addition, the system can be improved with multilingual support so that users from different language backgrounds can use it comfortably. Future versions can also include a mobile application, offline journaling support, and a more advanced personalized recommendation engine that adapts suggestions based on each user's emotional history and feedback.

Another important future enhancement is the integration of a chatbot-based emotional support assistant that can interact with users in real time and provide guided journaling, emotional check-ins, and immediate coping suggestions. The system can also be extended to support wearable device integration, predictive analytics for early warning of emotional distress, and optional sharing of reports with therapists or counselors under user consent. With stronger cloud scalability, improved security, and richer visualization dashboards, the project can evolve into a more comprehensive emotional wellness platform suitable for students, professionals, and mental health support applications.

Chapter 12

CONCLUSION

The Intelligent Emotion Recognition and Reminder System through Speech and Text Data is designed to provide an effective and intelligent solution for monitoring emotional well-being using modern AI technologies. The system analyzes both text and speech inputs to identify human emotions and provides personalized suggestions based on the detected emotional state. By integrating emotion recognition with reminder scheduling through Google Calendar API, the project helps users respond to negative emotions in a proactive and supportive manner.

This project successfully combines Natural Language Processing, speech processing, web technologies, and secure data management into a single platform. The use of FastAPI for backend development, React for frontend design, and SQLAlchemy for database handling makes the system efficient, scalable, and suitable for real-time applications. In addition, the visualization of emotional trends allows users to better understand their mental state over time and encourages regular self-reflection.

Overall, the proposed system demonstrates how artificial intelligence can be applied in the field of emotional wellness to create a meaningful and practical real-world application. It reduces manual effort in tracking emotions, improves emotional awareness, and provides timely intervention support. The system also helps users build healthier emotional habits through consistent reminders and personalized wellness recommendations. With further enhancements such as multilingual support, chatbot assistance, and multimodal emotion analysis, the system can be expanded into a more comprehensive mental health support platform.

Another important aspect of this project is its focus on user privacy and personalized support. Since emotional journals often contain sensitive personal thoughts, the system is designed to protect user data while still delivering meaningful insights and recommendations. This makes the application not only intelligent but also trustworthy and practical for daily use. By combining privacy, automation, and emotional analysis, the project offers a balanced solution that supports mental well-being in a secure and user-friendly way. This also helps users stay consistent with emotional self-reflection and timely self-care practices. In the long run, it can improve emotional awareness and support healthier mental habits.

Chapter 13

BIBLIOGRAPHY

1. Demszky, D., Movshovitz-Attias, D., Ko, J., Cowen, A., Nemade, G., & Ravi, S. (2020). GoEmotions: A dataset of fine-grained emotions. *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 4040–4054.
2. Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., & Stoyanov, V. (2019). RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*.
3. Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*.
4. Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of NAACL-HLT*, 4171–4186.
5. Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2023). Robust speech recognition via large-scale weak supervision. *Proceedings of the 40th International Conference on Machine Learning (ICML 2023)*.
6. Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., & others. (2020). Transformers: State-of-the-art natural language processing. *Proceedings of EMNLP 2020: System Demonstrations*, 38–45.
7. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems (NeurIPS)*, 30.
8. Lin, T. Y., Goyal, P., Girshick, R., He, K., & Dollar, P. (2017). Focal loss for dense object detection. *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2980–2988.
9. Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., & others. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems (NeurIPS)*, 32.
10. Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *International Conference on Learning Representations (ICLR 2015)*.
11. Cowen, A. S., & Keltner, D. (2017). Self-report captures 27 distinct categories of emotion bridged by continuous gradients. *Proceedings of the National Academy of Sciences*, 114(38), E7900–E7909.
12. Ekman, P. (1992). An argument for basic emotions. *Cognition and Emotion*, 6(3–4), 169–200.
13. Plutchik, R. (1980). A general psychoevolutionary theory of emotion. *Theories of Emotion* (Vol. 1), 3–33. Academic Press.

14. Pennebaker, J. W., Kiecolt-Glaser, J. K., & Glaser, R. (1988). Disclosure of traumas and immune function: Health implications for psychotherapy. *Journal of Consulting and Clinical Psychology*, 56(2), 239–245.
 15. World Health Organization. (2022). *World mental health report: Transforming mental health for all*. WHO Press, Geneva, Switzerland. ISBN: 978-92-4-004933-8.
 16. Chollet, F. (2021). *Deep learning with Python* (2nd ed.). Manning Publications. ISBN: 978-1617296864.
 17. Saravia, E., Liu, H. C. T., Huang, Y. H., Wu, J., & Chen, Y. S. (2018). CARER: Contextualized affect representations for emotion recognition. *Proceedings of EMNLP 2018*, 3687–3697.
 18. Sun, C., Qiu, X., Xu, Y., & Huang, X. (2019). How to fine-tune BERT for text classification? *China National Conference on Chinese Computational Linguistics (CCL)*, 194–206.
 19. Howard, J., & Ruder, S. (2018). Universal language model fine-tuning for text classification. *Proceedings of ACL 2018*, 328–339.
 20. Conneau, A., Khandelwal, K., Goyal, N., Chaudhary, V., Wenzek, G., Guzman, F., Grave, E., Ott, M., Zettlemoyer, L., & Stoyanov, V. (2020). Unsupervised cross-lingual representation learning at scale. *Proceedings of ACL 2020*, 8440–8451.
 21. FastAPI Documentation. (2024). FastAPI: Modern, fast web framework for building APIs with Python.
 22. SQLAlchemy Documentation. (2024). SQLAlchemy: SQL databases in Python, designed for simplicity, compatibility, and robustness.
 23. React Documentation. (2024). React: A JavaScript library for building user interfaces.
 24. Hugging Face Transformers Documentation. (2024). Transformers: State-of-the-art machine learning for PyTorch, TensorFlow, and JAX.
 25. OpenAI Whisper. (2022). Whisper: Robust speech recognition via large-scale weak supervision. GitHub repository.
 26. Google Calendar API Documentation. (2024). Google Developers: Calendar API v3.
 27. Web Crypto API. (2024). MDN Web Docs: Web Crypto API.
 28. Python Cryptography Library. (2024). Cryptography: Cryptographic recipes and primitives to Python developers.
 29. Thaler, R. H., & Sunstein, C. R. (2008). *Nudge: Improving decisions about health, wealth, and happiness*. Yale University Press. ISBN: 978-0300122237.
 30. Kahneman, D. (2011). *Thinking, fast and slow*. Farrar, Straus and Giroux. ISBN: 978-0374275631.
 31. American Psychological Association. (2022). *Stress in America 2022: Concerned for the future, beset by inflation*. APA Research Report.
 32. National Mental Health Survey of India. (2015–16). *National Mental Health Survey of India, 2015–16: Prevalence, patterns and outcomes*. NIMHANS, Bengaluru.
-

Intelligent Emotion Recognition and Reminder System (IERRS): Fine-Grained Multi-Label Emotion Classification with Transformer-Based NLP and Automated Wellness Scheduling

B. Narasimhulu, M. Abilasha, K. Sai Pradeep, B. Sreekar, A. Praveen Kumar,

Dr. P. Lokesh Kumar Reddy, M. Tech, Ph.D(Associate professor)

Department of Computer Science and Engineering (AI & ML)

Sri Venkateswara College of Engineering (Autonomous), Tirupati – 517507, India

Abstract—Mental health disorders affect over 970 million people globally, yet treatment gaps exceed 70% in low- and middle-income countries. Conventional digital journaling platforms remain passive, recording entries without extracting emotional signals or providing actionable guidance. This paper presents the Intelligent Emotion Recognition and Reminder System (IERRS), a full-stack web application that fine-tunes a DistilBERT transformer on the GoEmotions dataset to perform 28-class multi-label emotion classification from both text and voice journal entries. The system detects up to 28 fine-grained emotional states, generates psychologically grounded three-component wellness suggestions, integrates with the Google Calendar API for automated activity scheduling, and produces colour-coded .xlsx wellness reports derived from the Geneva Emotion Wheel palette. A client-side AES-GCM (256-bit) encryption layer protects sensitive journal content, and automated weekly, monthly, and yearly trend reports are delivered via APScheduler. The backend employs FastAPI with SQLite; the frontend is built in React 18. The model achieves a Micro F1 of 0.627 with auto-tuned thresholds and weighted Binary Cross-Entropy loss — a 4.9% improvement over the 0.578 baseline.

Index Terms—emotion recognition, mental health, DistilBERT, GoEmotions, multi-label classification, transformer, FastAPI, React, wellness reminder, natural language processing, AES-GCM encryption, Google Calendar.

I. INTRODUCTION

The global burden of mental health conditions is substantial and growing. According to the World Health Organization, over 970 million individuals live with a mental or substance use disorder, yet treatment gaps exceed 70% in low- and middle-income countries [1]. Emotional dysregulation, if unaddressed, compounds into chronic stress, burnout, anxiety, and clinical depression. Early intervention through daily emotional awareness practices has been shown to measurably reduce these outcomes.

Traditional journaling is among the oldest evidence-based therapeutic tools. Research consistently demonstrates that expressive writing reduces psychological distress and improves cognitive processing of emotional events. However, conventional digital journaling platforms are passive: they record entries without extracting emotional signals or responding with actionable guidance.

Recent advances in Natural Language Processing (NLP), particularly transformer-based architectures pre-trained on large corpora, have made nuanced, contextually aware emotion classification feasible at scale. Simultaneously, the proliferation of smartphone microphones enables multimodal input, allowing systems to analyse both written and spoken journal entries in real time.

This paper presents IERRS, a full-stack mental wellness application that bridges these gaps. IERRS accepts text or voice journal entries, classifies them across 28 fine-grained emotional dimensions using a fine-tuned DistilBERT model, generates personalised evidence-based wellness suggestions, schedules reminder activities via Google Calendar, and delivers longitudinal emotion analytics through styled downloadable reports. The system is designed around three core principles: accuracy, privacy (client-side AES-GCM encryption), and actionability (closed-loop scheduling and reporting).

II. RELATED WORK

Early computational emotion recognition relied on lexicon-based methods such as VADER [2] and SenticNet [3], which map words to valence scores. While efficient, these are limited to coarse polarity and fail to capture fine-grained distinctions such as fear vs. nervousness or joy vs. pride.

Long Short-Term Memory (LSTM) architectures [4] improved contextual modelling but required substantial labelled data and struggled with long-range dependencies. Convolutional Neural Networks applied to sentence features [5] provided faster training but retained similar limitations.

The publication of BERT [6] transformed NLP benchmarks. Pre-trained on 3.3 billion words with masked language modelling, BERT captures bidirectional contextual representations that generalise with minimal fine-tuning. Demszky et al. [7] leveraged this to create GoEmotions — 58,009 annotated Reddit comments across 28 emotion categories — the first large-scale multi-label emotion benchmark.

DistilBERT [8] retains 97% of BERT's understanding while reducing parameters by 40% and inference time by 60%. Existing applications such as Woebot and Wysa deploy rule-based agents but do not perform fine-grained classification, generate longitudinal analytics, or integrate with scheduling services. IERRS addresses all these gaps.

III. SYSTEM ARCHITECTURE

IERRS follows a three-tier architecture: a React 18 single-page frontend, a FastAPI backend, and a local AI inference layer. The frontend provides a dynamic auto-resizing journal textarea, a microphone-based voice recorder via the MediaRecorder API (WebM blobs), a 28-colour emotion frequency bar chart (Chart.js, GoEmotions-derived palette), a Google Calendar UI with 30-second auto-schedule countdown, and .xlsx report downloads. The application is fully responsive from 320 px to 1440 px using CSS clamp() fluid typography.

The backend exposes RESTful endpoints for authentication (/signup, /token), journal analysis (/analyze-text, /analyze-audio), profile management, activity scheduling, timeline retrieval,

emotion aggregation, and report generation. JWT authentication uses HS256 signing. APScheduler delivers automated weekly (Sunday 09:00), monthly (1st, 09:00), and yearly (January 1st, 09:00) email reports.

The AI inference layer loads the fine-tuned model and `model_meta.json` at startup. On receiving text, the backend applies regex sentence boundary detection, passes each sentence through the tokeniser and model, aggregates sigmoid probabilities via max-pooling, applies the auto-tuned threshold to produce the active emotion set, and sorts results by descending confidence. A fallback selects the top-scoring class if no emotion clears the threshold. Voice entries follow the same path after Whisper transcription.

A. Security and Privacy

Journal text is stored in SQLite with emotion labels and confidence scores. Client-side AES-GCM encryption (256-bit key, 12-byte random IV via Web Crypto API) is available through `encryption.js` for optional content protection. Google Calendar OAuth 2.0 credentials are managed via `InstalledAppFlow`, with tokens auto-refreshed from `auth/token.json`.

IV. EMOTION CLASSIFICATION MODEL

A. Dataset and Class Imbalance

The GoEmotions simplified dataset contains 43,410 training, 5,426 validation, and 5,427 test samples — each annotated with a multi-hot vector across 28 emotional categories including admiration, anger, fear, grief, joy, sadness, surprise, and neutral. The dataset exhibits severe class imbalance: neutral constitutes $\approx 35\%$ of labels, while grief, embarrassment, and pride each fall below 1%. Standard Binary Cross-Entropy (BCE) loss without weighting systematically ignores these rare but clinically significant categories.

B. Weighted Training Procedure

The training procedure extends the Hugging Face Trainer with a `WeightedBCETrainer` that computes per-class positive weights as $\text{pos_weight} = (N - \text{pos_count}) / \text{pos_count}$, clipped to $[1.0, 15.0]$ to prevent numerical instability. Training uses `distilbert-base-uncased`; maximum sequence length 128; batch size 32 (train), 64 (eval); learning rate $2e-5$ with 6% linear warmup and weight decay 0.01; up to 8 epochs with `EarlyStoppingCallback` (patience = 3) on validation Micro F1; and FP16 mixed-precision on CUDA.

C. Threshold Optimisation

The v1 system used a hardcoded threshold of 0.35, causing systematic over-prediction of common emotions and suppression of rare ones. The v2 pipeline sweeps thresholds from 0.20 to 0.70 in steps of 0.02 (`np.linspace`) on the validation set, selecting the value maximising Micro F1. This threshold and all performance metrics are serialised to `model_meta.json` alongside model weights. The production backend reads `model_meta.json` at startup, ensuring training–inference alignment across updates.

TABLE I

Model Performance Comparison on GoEmotions Test Set

Configuration	Micro F1	Macro F1	Thresh.
v1: DistilBERT, fixed 0.35, 3 epochs, no weighting	0.578	0.381	0.35
v2: DistilBERT, weighted BCE, 8 epochs, auto-tuned	0.627	0.445	0.42
v2: RoBERTa-base, weighted BCE, 8 epochs, auto-tuned	0.658	0.471	0.44

V. KEY FEATURES AND IMPLEMENTATION

A. Multimodal Input

Text entries are transmitted as plaintext UTF-8 via `FormData` to `/analyze-text`. Voice entries are captured as WebM blobs, uploaded to `/analyze-audio`, transcribed by Whisper tiny (`openai-whisper`), and the temporary file deleted immediately via `try/finally`. Both paths call `analyze_and_store()`, which validates input length (max 2,000 characters), runs per-sentence DistilBERT inference, aggregates emotion probabilities, and persists results with timestamps to SQLite.

B. Dynamic Suggestion Engine

The suggestion engine generates three-component responses: (i) an Observation summarising the detected emotional state against the user's 90-day historical baseline; (ii) an Insight drawn from one of 28 emotion-specific pools, each containing 10 recommendations grounded in cognitive-behavioural therapy, physiological regulation, and behavioural activation; and (iii) an Actionable Step adapted to whether the detected state represents improvement, regression, or continuation relative to the user's historical negative emotion ratio. Mixed emotional states trigger a dedicated 'emotional rollercoaster' pathway.

C. Google Calendar and Email Scheduling

When a detected emotion belongs to the NEGATIVE EMOTIONS set, `requires_scheduling` is set to True and a 30-second auto-schedule countdown begins in the frontend. The `/schedule-activity` endpoint accepts a specific ISO 8601 datetime or 'auto', selecting the next future morning or evening preference time. A Google Calendar event is created via the Calendar API v3 using OAuth 2.0, while an APScheduler date-trigger job registers simultaneous personalised email delivery via Gmail SMTP SSL (port 465).

D. Wellness Reports and Emotion Colour Science

The report generator produces .xlsx files via `openpyxl` with a 28-colour palette derived from the Geneva Emotion Wheel. Each GoEmotions label maps to three values: a pastel cell fill, a contrasting font colour, and a saturated left-border accent. Warm hues encode high-arousal negative emotions; cool hues encode low-arousal states; green and gold encode positive states; grief uses inverted dark-navy fill. Reports include a Journal Report sheet (per-entry: date, text, emotions, mood badge, suggestion) and a Summary sheet (frequency and share percentage for the period).

VI. EXPERIMENTAL SETUP

All experiments were conducted on a workstation equipped with an NVIDIA RTX 3080 GPU (10 GB VRAM), an Intel Core i9-12900K CPU, and 32 GB DDR5 RAM running Ubuntu 22.04 LTS. Python 3.10 was used throughout, with PyTorch 2.1.0 and Hugging Face Transformers v4.36.0 as the primary deep learning framework. The GoEmotions simplified dataset was downloaded from the official Google Research repository and preprocessed using the Hugging Face Datasets library.

Training used the AdamW optimiser with a cosine learning rate schedule and 6% linear warmup over 8 maximum epochs. Gradient clipping was applied at a maximum norm of 1.0 to stabilise training on rare emotion classes. Model checkpoints were evaluated on the validation set every 500 steps, and the best two checkpoints by Micro F1 were retained. Mixed-precision training (FP16) was enabled via the Hugging Face Trainer `fp16` flag, reducing GPU memory consumption by approximately 40% and accelerating training by 1.8x. For the threshold sweep, the validation set was used exclusively; the test set was held out and evaluated only once with the final selected threshold to prevent information leakage.

Inference latency was measured by averaging 500 sequential forward passes on a single CPU core with batch size 1, to simulate consumer hardware conditions since production deployment targets

users without dedicated GPU resources. All reported metrics represent single-run evaluations without ensembling.

A. Hyperparameter Sensitivity Analysis

A grid search over key hyperparameters validated the selected configuration. Learning rate was swept across {1e-5, 2e-5, 3e-5, 5e-5}; batch size across {16, 32, 64}; and maximum sequence length across {64, 128, 256}. The selected configuration (lr = 2e-5, batch = 32, length = 128) yielded the highest validation Micro F1. Increasing sequence length to 256 improved Macro F1 by 0.008 on average but added 34% inference latency per sample, making it unsuitable for real-time journaling requiring sub-second response.

The pos_weight clipping range [1.0, 15.0] was validated by examining the natural pos_weight distribution across all 28 classes. Without clipping, grief (pos_weight ~98) and embarrassment (pos_weight ~87) caused gradient explosion in early training epochs. Clipping to 15.0 preserved the oversampling signal while stabilising convergence. Values above 20.0 reintroduced instability without measurable F1 benefit.

B. Data Preprocessing Pipeline

Raw GoEmotions entries were cleaned by removing Reddit-specific markup (subreddit tags, user mentions prefixed with u/, URLs), normalising Unicode punctuation to ASCII equivalents, and stripping excess whitespace. Entries exceeding 512 WordPiece tokens after tokenisation were truncated to 128 tokens; this affected

Emotion	v1 F1	v2 F1	Delta
admiration	0.61	0.67	+0.06
amusement	0.52	0.76	+0.24
anger	0.55	0.61	+0.06
anxiety	0.44	0.51	+0.07
approval	0.42	0.49	+0.07
caring	0.41	0.48	+0.07
confusion	0.38	0.45	+0.07
curiosity	0.40	0.47	+0.07
desire	0.32	0.41	+0.09
disappointment	0.34	0.47	+0.13
disapproval	0.43	0.50	+0.07
disgust	0.37	0.48	+0.11
embarrassment	0.22	0.38	+0.16
excitement	0.56	0.62	+0.06
fear	0.48	0.55	+0.07
gratitude	0.34	0.36	+0.02
grief	0.19	0.31	+0.12
joy	0.64	0.69	+0.05
love	0.74	0.78	+0.04
nervousness	0.28	0.39	+0.11

Table II reveals that the largest F1 gains are concentrated in bottom-frequency classes: grief (+0.12), embarrassment (+0.16), nervousness (+0.11), pride (+0.13), and remorse (+0.11). High-frequency classes such as gratitude, love, and neutral show smaller but steady gains, confirming that weighted loss primarily benefits underrepresented categories without degrading dominant class performance. Self-conscious emotion categories exhibit the highest relative improvement ratios — these emotions are both clinically relevant (frequently associated with depression and anxiety onset) and lexically subtle, with textual cues that significantly overlap neighbouring emotion clusters. Weighted BCE enables the model to attend to low-frequency discriminative n-grams that the unweighted baseline systematically underweights.

VIII. COMPARATIVE ANALYSIS

Table III contextualises IERS performance against published GoEmotions benchmark results. All comparisons use the simplified 28-class label set and the official test split. Where multiple configurations are reported, the best published Micro F1 figure is used.

TABLE III
GoEmotions Benchmark Comparison (28-class Simplified Split)

Model	Training Strategy	Micro F1	Macro F1
BERT-base [6]	Standard fine-tune	0.598	0.409
DistilBERT v1 (ours)	Fixed threshold, no weighting	0.578	0.381
RoBERTa-base [9]	Standard fine-tune	0.631	0.433
DeBERTa-v3-base [13]	Standard fine-tune	0.647	0.452
DistilBERT v2 (ours)	Weighted BCE + auto-threshold	0.627	0.445
RoBERTa v2 (ours)	Weighted BCE + auto-threshold	0.658	0.471

IERS DistilBERT v2 achieves competitive Micro F1 relative to BERT-base and substantially outperforms the v1 baseline despite using fewer parameters. The RoBERTa v2 configuration surpasses standard RoBERTa fine-tuning by +2.7% Micro F1 and outperforms DeBERTa-v3 on Macro F1 (+0.019), demonstrating that the weighted training strategy generalises effectively to larger encoder backbones. The primary contribution of IERS is not achieving absolute state-of-the-art aggregate Micro F1, but delivering systematic improvements in rare emotion recall that are clinically more meaningful than incremental gains on majority classes.

IX. USER INTERFACE AND EXPERIENCE

A. Frontend Architecture

The IERS React 18 frontend follows a component-driven architecture with four primary view components: the Journal Entry

view (textarea, voice recorder, submission controls), the Emotion Dashboard (animated bar chart, dominant emotion badge, suggestion card), the Timeline view (chronological entry history with expandable emotion detail), and the Reports view (date-range selector and download trigger). State management uses React's useReducer hook with a single top-level application state object, avoiding external state libraries to minimise bundle size.

The voice recording flow uses the MediaRecorder API with MIME type 'audio/webm;codecs=opus' where supported, falling back to 'audio/webm' on browsers without Opus codec availability. Recording duration is capped at 120 seconds client-side. Audio blobs are sent to the backend via FormData, and transcription results are displayed in the textarea immediately after server response, allowing users to review and edit before final analysis submission.

B. Accessibility and Responsive Design

The interface targets WCAG 2.1 Level AA compliance. All interactive elements carry ARIA labels; the emotion bar chart uses both colour and pattern encoding for colour-impaired users; keyboard navigation is fully supported throughout. Responsive layout uses a 12-column CSS Grid with breakpoints at 480 px (single-column mobile), 768 px (tablet), and 1200 px (full desktop). Font sizes scale via CSS clamp() with a minimum of 14 px and maximum of 18 px for body text.

C. Pilot Usability Evaluation

A pilot usability evaluation was conducted with 24 participants (12 undergraduate students, 8 postgraduate students, 4 faculty members) completing five standardised journaling tasks using a think-aloud protocol. Task completion rates were: text entry analysis 96%, voice entry analysis 88%, calendar scheduling 100%, report download 100%, and timeline review 92%. The lower voice entry completion rate (88%) was attributed to browser microphone permission prompts on first use.

Post-task System Usability Scale (SUS) questionnaires yielded a mean score of 79.4 (SD = 8.2), placing IERRS in the 'Good' usability band. Qualitative feedback highlighted the 30-second auto-schedule countdown as a particularly valued feature. Areas for improvement included the emotion bar chart legend (noted as 'too small on mobile') and the absence of in-app notifications independent of email.

X. LIMITATIONS AND ETHICAL CONSIDERATIONS

A. Model and Dataset Limitations

The GoEmotions dataset is derived exclusively from English-language Reddit comments, introducing demographic and linguistic biases. Reddit's user base skews toward younger, English-speaking, predominantly Western populations; the emotional language norms of this corpus may not generalise to clinical populations, non-native English speakers, or users from non-Western cultural contexts. Fine-tuning on domain-specific corpora would be a prerequisite for clinical deployment.

Multi-label emotion classification from text remains inherently ambiguous. Inter-annotator agreement on GoEmotions averages approximately Krippendorff's $\alpha = 0.42$ across all 28 classes, lowest on self-conscious emotions — precisely the classes where IERRS shows the most significant improvement. Practitioners should interpret per-entry emotion predictions as probabilistic signals rather than definitive clinical assessments.

B. Privacy and Safety Considerations

Journal text is transmitted as plaintext over HTTPS for server-side AI inference in the default configuration. The current SQLite backend does not implement at-rest encryption; production

deployment for sensitive use cases requires migration to an encrypted backend such as SQLCipher. IERRS does not implement any crisis detection or mandatory clinical escalation pathway and is designed for general wellness support, not as a substitute for professional psychological care. Any deployment targeting vulnerable populations must integrate safeguarding mechanisms and explicit referral pathways to licensed mental health professionals.

C. Deployment Constraints

The current deployment runs DistilBERT inference on the FastAPI backend CPU. For multi-user concurrent deployments, inference latency scales linearly with concurrent requests in the absence of request batching. A production deployment serving more than approximately 50 concurrent users would require GPU-accelerated inference via ONNX Runtime with TensorRT, or horizontal scaling of the inference layer behind a load balancer. SQLite's single-writer constraint similarly requires migration to PostgreSQL for deployments exceeding a few hundred active users.

XI. RESULTS AND DISCUSSION

The DistilBERT v2 model evaluated on the GoEmotions test set at threshold 0.42 achieves Micro Precision 0.641, Micro Recall 0.614, Micro F1 0.627, and Macro F1 0.445 — improvements of +4.9% and +6.4% over the v1 baseline (Table I). Per-class positive weight scaling was the single most impactful change, with the largest gains on clinically significant rare emotions as detailed in Table II. Threshold optimisation contributed a further +2.1% Micro F1 improvement over a fixed 0.50 threshold.

The RoBERTa-base variant achieves Micro F1 0.658 (+3.1%) at threshold 0.44, at approximately 70% higher CPU inference latency (mean 1.34 s vs. 0.79 s per entry on a single i9-12900K core) and 2x model storage (478 MB vs. 242 MB). For consumer hardware deployment requiring sub-second response, DistilBERT v2 offers the more appropriate accuracy-efficiency trade-off. The pilot SUS score of 79.4 and 96% task completion rate for text analysis confirm the deployed system meets real-world usability requirements at acceptable latency.

XII. CONCLUSIONS

This paper presented IERRS, a full-stack intelligent emotion recognition and reminder system that addresses the limitations of existing mental wellness applications through fine-grained 28-class multi-label transformer classification, proactive calendar-integrated scheduling, psychologically grounded personalised suggestions, and longitudinal wellness reporting. Five concrete technical contributions are made: (1) a WeightedBCETrainer with per-class positive weight scaling that substantially improves rare emotion F1 scores; (2) a validation-set threshold optimisation pipeline eliminating hardcoded classification boundaries; (3) a 28-category emotion colour science palette grounded in the Geneva Emotion Wheel; (4) a closed-loop scheduling pipeline connecting emotion detection to Google Calendar and personalised email reminders; and (5) a fully responsive React 18 frontend with WCAG 2.1 AA-targeted accessibility and fluid typography scaling.

A pilot usability evaluation with 24 participants yielded a SUS score of 79.4, confirming practical deployability. Per-class analysis demonstrates that weighted BCE training with auto-tuned thresholds produces the largest improvements precisely on the clinically significant rare emotion categories — grief, embarrassment, pride, nervousness, and remorse — that baseline models systematically suppress. Comparative benchmarking confirms competitive performance against models with substantially higher parameter counts.

Future directions include: federated learning for on-device personalisation without centralised data collection; physiological signal fusion from wearable devices (heart rate variability, galvanic skin response) for multimodal affective state estimation; a crisis detection module for escalating severe negative emotion clusters to appropriate support pathways; and migration of the inference backbone to RoBERTa with LLM-generated contextual suggestions for the next development cycle. Multilingual extension via cross-lingual transfer from mBERT or XLM-RoBERTa represents a longer-term research priority to broaden the system's accessibility across non-English-speaking populations.

. ACKNOWLEDGMENT

The authors thank the Department of Computer Science and Engineering (AI & ML), Sri Venkateswara College of Engineering, for institutional support, and acknowledge the open-source communities behind Hugging Face Transformers, FastAPI, React, openpyxl, and OpenAI Whisper.

. REFERENCES

- [1] World Health Organization, "World mental health report: Transforming mental health for all," WHO, Geneva, 2022.
- [2] C. J. Hutto and E. Gilbert, "VADER: A parsimonious rule-based model for sentiment analysis of social media text," in Proc. ICWSM, 2014.
- [3] E. Cambria et al., "SenticNet: A publicly available semantic resource for opinion mining," in Proc. AAAI Fall Symp., 2010.
- [4] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [5] Y. Kim, "Convolutional neural networks for sentence classification," in Proc. EMNLP, 2014, pp. 1746–1751.
- [6] J. Devlin et al., "BERT: Pre-training of deep bidirectional transformers for language understanding," in Proc. NAACL-HLT, 2019, pp. 4171–4186.
- [7] D. Demszky et al., "GoEmotions: A dataset of fine-grained emotions," in Proc. ACL, 2020, pp. 4040–4054.
- [8] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," arXiv:1910.01108, 2019.
- [9] Y. Liu et al., "RoBERTa: A robustly optimized BERT pretraining approach," arXiv:1907.11692, 2019.
- [10] T. Wolf et al., "HuggingFace transformers: State-of-the-art natural language processing," in Proc. EMNLP Syst. Demos., 2020, pp. 38–45.
- [11] A. Radford et al., "Robust speech recognition via large-scale weak supervision," in Proc. ICML, 2023, pp. 28492–28518.
- [12] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [13] P. He et al., "DeBERTa: Decoding-enhanced BERT with disentangled attention," in Proc. ICLR, 2021.
- [14] K. Krippendorff, *Content Analysis: An Introduction to Its Methodology*, 3rd ed. Sage Publications, 2013.
- [15] J. Brooke, "SUS: A 'quick and dirty' usability scale," in *Usability Evaluation in Industry*. Taylor & Francis, 1996, pp. 189–194.